



HipsterShop Platform — Complete DevOps Engineering Documentation

Organization: HipsterShopp | **Repositories:** 14 | **Status:** Production | **Last Updated:** April 2026



Table of Contents

#	Section
1	Executive Summary
2	Business Requirements & Problem Statement
3	Architecture Overview — HLD
4	Detailed Design — LLD (All 12 Services)
5	Infrastructure Setup
6	Kubernetes Implementation Deep Dive
7	Helm-Based Deployment Strategy
8	CI/CD Pipeline
9	GitOps Deployment — ArgoCD
10	Deployment Strategies
11	Security Architecture
12	Observability
13	Performance & Scalability
14	Networking Architecture
15	Configuration Management
16	12-Factor App Compliance
17	Disaster Recovery & Backup
18	Runbook — Operations Guide
19	Troubleshooting Guide
20	Challenges & Solutions

21	Architecture Decision Records
22	Environment Strategy
23	Release & Versioning Strategy
24	Incident Management & SLA
25	Non-Functional Requirements
26	Cost & Resource Optimization
27	Future Enhancements
28	Appendix — Full Command Reference

1. Executive Summary

HipsterShop is a production-grade, cloud-native e-commerce platform composed of **12 independently deployable microservices** spanning Go, Java, Python, Node.js, and .NET — all running on Kubernetes, managed via **ArgoCD GitOps**, deployed through **GitHub Actions CI/CD**, and versioned with **Helm charts**.

Users browse products, authenticate with JWT, manage carts, place orders, process payments, receive email confirmations, and interact with an **AI shopping assistant** powered by Google Gemini 2.5 Flash.

Core Technology Stack

Layer	Technology	Why Used
 Container Runtime	Docker (multi-stage builds)	Package each service with its exact runtime; eliminate environment drift across 5 languages
 Orchestration	Kubernetes (kubeadm cluster)	Self-healing, rolling updates, horizontal scaling, service discovery
 Package Manager	Helm v3	Eliminate 24+ manually written YAMLS; centralize config; per-environment overrides
 CI Pipeline	GitHub Actions (shared reusable workflows)	One shared library powers all 12 services; change once, propagate everywhere
 CD / GitOps	Argo CD (ApplicationSet-driven)	Git as single source of truth; drift detection; declarative reconciliation
 Progressive Delivery	Argo Rollouts (frontend blue-green)	Zero-downtime frontend deployments with manual promotion gate
 Secret Encryption	Bitnami Sealed Secrets	Encrypt Kubernetes Secrets in Git; safe to commit; cluster-specific decryption

 API Gateway	kGateway (Kubernetes Gateway API)	Single entry point; path-based routing; eliminates per-service LoadBalancers
 Database	MongoDB 7.0 (3-node Replica Set)	Persistent data; SCRAM-SHA-256 auth; per-service isolated databases
 Persistent Storage	NFS StorageClass	Dynamic PVC provisioning for MongoDB; pod-reschedulable without data loss
 AI/ML	Google Gemini 2.5 Flash (LangGraph)	AI shopping assistant with agentic tool use (product fetch + cart add)
 Tracing	Jaeger (via OpenTelemetry)	Distributed tracing across all services
 OIDC Auth	GitHub OIDC (shippingservice)	Zero-stored-credential production deploy; short-lived JWT

Key Outcomes

-  **Automated CI/CD** — code push to running pod in ~10–15 minutes
-  **Zero/near-zero downtime** — rolling updates + blue-green for frontend
-  **GitOps** — git is the only source of truth; ArgoCD selfHeal reverts manual changes
-  **Keyless production deployment** — ShippingService deployed via GitHub OIDC JWT (no stored kubeconfig)
-  **Security by default** — Network policies, Sealed Secrets, RBAC, distroless images
-  **12 microservices, 14 repos** — loosely coupled, independently deployable

2. Business Requirements & Problem Statement

Why Microservices for E-Commerce

Monolith Problems:

- Scale the entire app when only one service is stressed (e.g., product catalog during flash sales)
- Bug in payment service requires full application redeployment
- All teams blocked on a single deployment pipeline
- Technology lock-in — one language/framework for everything

Microservices Solutions:

Business Requirement	Microservices Solution
Independent scaling	Scale only <code>productcatalogservice</code> during browse spikes — other services unaffected
Fault isolation	Payment failure doesn't bring down catalog browsing
Faster release cycles	Each service has its own CI/CD pipeline; teams deploy independently

Technology flexibility	Java for JVM ecosystem (adservice), Go for performance (auth, checkout, shipping), Python for AI/ML
Security isolation	Separate MongoDB databases per service; network policies prevent cross-service DB queries
Auditability	Per-service logs, separate git history per service repo

Platform Goals

- 1. **High Availability** — 3-node MongoDB replica set; rolling updates; ArgoCD drift healing
 - 2. **Security** — Sealed Secrets; Network Policies (default-deny); RBAC; OIDC; distroless images
 - 3. **Developer Velocity** — One shared CI library; `git push` triggers full automation; ArgoCD auto-syncs dev
 - 4. **Observability** — OpenTelemetry → Jaeger for distributed tracing; structured JSON logs to stdout
 - 5. **Elasticity** — HPA on frontend (1–5 replicas); resource limits on all 12 services
-

3. Architecture Overview — HLD

3.1 High-Level Architecture Diagram

EXTERNAL USERS (HTTP)

| Port 80



kGateway (Kubernetes Gateway API – kgateway controller)

GatewayClass: kgateway | hipstershop-gateway | Port :80

11 HTTPRoute objects with URLRewrite filters

Namespace: hipster-backend (allowedRoutes: from: All)

/	/api							Each path →
▼	▼	▼	▼	▼	▼	▼	▼	▼ ClusterIP:port

Namespace: hipster-frontend

frontend Go :8080 (Service port :80)

PROD: Argo Rollouts Blue-Green

frontend-active (live traffic)

frontend-preview (staging, manual promote)

scaleDownDelaySeconds: 60 | autoPromotionEnabled: false

HPA: min=1, max=5 (CPU: 70%, Memory: 80%)

| calls all backend services



Namespace: hipster-backend

adservice	Java 25 / Gradle	:9555	Ads (OTel agent, GHCR prod)
authservice	Go 1.26	:8081	JWT auth (gorilla/mux + logrus)
cartservice	.NET 8 / C#	:7070	Cart CRUD (ASP.NET Core, USER 1000)
checkoutservice	Go 1.26	:5050	Checkout orchestrator (5 services)
currencyservice	Node.js 20	:7000	Currency conversion
emailservice	Python 3.11	:8080	SMTP order email (port 587 egress)
paymentservice	Node.js 20	:50051	Simulated charge + MongoDB persist
productcatalog	Go 1.26	:3550	Product REST API + MongoDB/JSON
recommendationservice	Python 3.11	:8080	Product recommendations
shippingservice	Go 1.26	:50051	Shipping quotes (OIDC prod deploy)
assistantservice	Python 3.10	:8080	Gemini 2.5 Flash AI (LangGraph)

ConfigMaps: hipster-config, service-addresses, service-ports,
mongodb-config, app-features

Secrets: app-secrets (JWT+Gemini), mongodb-root, mongodb-users

NetworkPolicies: default-deny-all, backend-allow, emailservice-allow-smtp

| MongoDB connections via headless DNS



Namespace: hipster-database

MongoDB StatefulSet | 3 replicas | mongo:7.0 | ReplicaSet: rs0
DNS: mongodb-{0,1,2}.mongo-headless.hipster-database.svc.cluster.local:27017
Auth: SCRAM-SHA-256 | Per-service users + isolated databases
Storage: NFS PVCs | 5Gi per replica = 15Gi total
Databases: auth_db, cart_db, catalog_db, payment_db + more

Namespace: argocd

ArgoCD server + repo-server + application-controller
AppProject: hipstershop | 3 ApplicationSets (main, dev, prod)
Sync Waves: 0=mongodb+backend-common, 1=sealed-secrets, 2=gateway, 3=micro-apps

3.2 CI/CD Flow Diagram

Developer git push / PR



GitHub Actions – per-service caller workflow



```
[PR only] sonar-and-snyk
          SonarQube Quality Gate + Snyk HIGH+ dep scan
```

(success or push event)

```
generate-tag-and-build-image-trivy
  main branch → service-vX.Y.Z (semver via conventional commits)
  develop    → develop-{sha8}
  docker build → saves image.tar artifact
  [PR] Trivy CRITICAL CVE scan
```

(push event only)

```
push
  docker push yampss/hipstershop-{service}:{tag}
  docker push :latest (main branch only)
  git tag (main branch only)
```

↓ develop branch

↓ main branch

```
update-helm-values-dev
(sed tag in values.dev.yaml
commit hipstershop-helm-charts:develop)
```

```
push-to-ghcr
(cross-push GHCR)
```

prod-approval-gate (human 🙋)

```
update-helm-values-prod
(sed tag in values.prod.yaml
commit hipstershop-helm-charts:main)
[shippingservice] deploy-to-prod
(OIDC kubectl set image)
```

```
ArgoCD polls helm-charts (~3 min)
micro-apps-appset-dev
auto-sync (prune:true)
```

```
ArgoCD polls helm-charts (~3 min)
micro-apps-appset-prod
auto-sync (prune:false)
```

Kubernetes rolling update
Dev Cluster

Kubernetes rolling update
Prod Cluster (Blue-Green for frontend)

Failure at any step → notify.yaml → SendGrid failure email

3.3 Dual-Cluster Strategy

Attribute	Dev Cluster	Prod Cluster
ApplicationSet	<code>micro-apps-appset-dev.yaml</code>	<code>micro-apps-appset-prod.yaml</code>
Branch tracked	<code>develop</code>	<code>main</code>
Values files	<code>values.yaml</code> + <code>values.dev.yaml</code>	<code>values.yaml</code> + <code>values.prod.yaml</code>
Prune	✔ Yes — fast convergence	✗ No — safety net (orphaned resources kept)
ServerSideApply	✔ Yes	✗ No — Rollouts conflict prevention
Argo Rollouts	—	✔ Frontend blue-green
ShippingService	Managed by ArgoCD	Direct OIDC <code>kubectl set image</code>
Services count	12	11 (shipping excluded from AppSet)

3.4 Repository Map

Repository	Language	Purpose	Connected To
<code>hipstershop-adservice</code>	Java 25 / Gradle	Personalized ads (GHCR prod)	helm-charts, shared-workflows
<code>hipstershop-authservice</code>	Go 1.26	JWT authentication + MongoDB	MongoDB, helm-charts
<code>hipstershop-cartservice</code>	.NET 8 / C#	Shopping cart + MongoDB	MongoDB, helm-charts
<code>hipstershop-checkoutservice</code>	Go 1.26	Checkout orchestration (5 deps)	payment, shipping, email, cart, currency
<code>hipstershop-currencyservice</code>	Node.js 20	Currency conversion	helm-charts
<code>hipstershop-emailservice</code>	Python 3.11	SMTP order confirmation	helm-charts (SMTP egress)
<code>hipstershop-frontend</code>	Go 1.26	HTML web UI + Argo Rollouts	All backend services
<code>hipstershop-paymentservice</code>	Node.js 20	Simulated payment + MongoDB	MongoDB, helm-charts
<code>hipstershop-productcatalogservice</code>	Go 1.26	Product catalog REST API	MongoDB, helm-charts
<code>hipstershop-recommendationservice</code>	Python 3.11	Product recommendations	productcatalog, helm-charts

hipstershop-shippingservice	Go 1.26	Shipping quotes (OIDC)	helm-charts
hipstershop-assistant-service	Python 3.10 / FastAPI	Gemini AI shopping assistant	productcatalog, cartservice via gateway
hipstershop-shared-workflows	YAML (GitHub Actions)	Reusable CI/CD library	All 12 service repos
hipstershop-helm-charts	Helm / YAML	GitOps control plane	ArgoCD in cluster

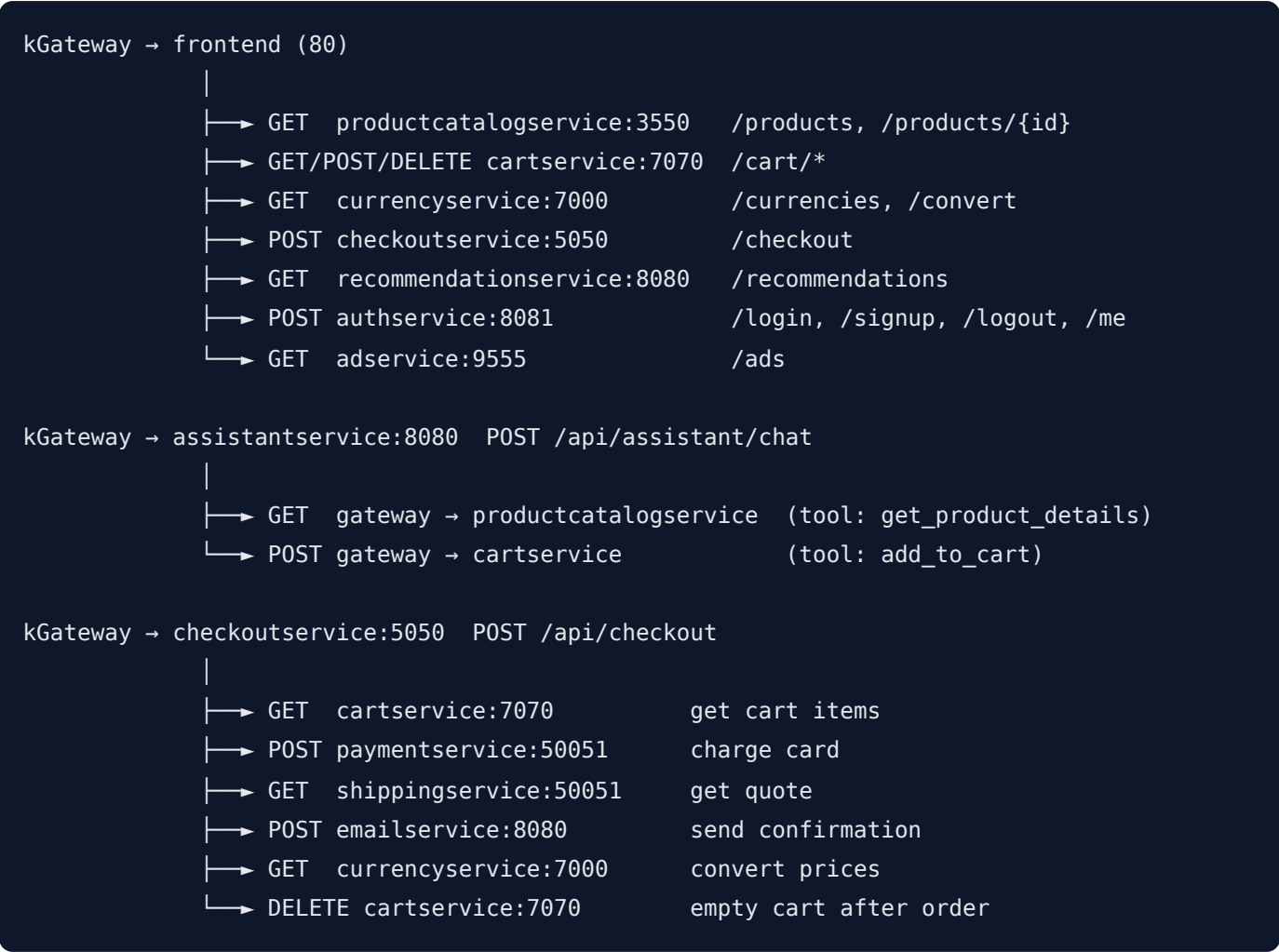
4. Detailed Design — LLD

4.1 Service Design Summary

Service	Language	Port	DB	Key Dependency	Why This Tech
adservice	Java 25	9555	None	OTel Java agent	JVM ecosystem for ad serving; preserves original demo design
authservice	Go 1.26	8081	MongoDB auth_db	gorilla/mux, logrus	Fast startup, static binary, distroless-compatible
cartservice	.NET 8	7070	MongoDB cart_db	ASP.NET Core	Demonstrates .NET in polyglot platform; built-in DI
checkoutservice	Go 1.26	5050	None	5 services	Go's concurrency model suits orchestration; custom money math
currencyservice	Node.js 20	7000	None	—	JavaScript suitable for data transformation APIs
emailservice	Python 3.11	8080	None	SMTP port 587	Python's smtplib simplicity; consistent with other Python services
frontend	Go 1.26	8080	None	All 7 backends	Go HTML templates; fast static binary; distroless
paymentservice	Node.js 20	50051	MongoDB payment_db	Express.js, charge.js	Node.js rapid API development; graceful DB degradation
productcatalogservice	Go 1.26	3550	MongoDB catalog_db	JSON fallback	Go REST + MongoDB driver; seed from embedded JSON

recommendationservice	Python 3.11	8080	None	productcatalog	Python ML-friendly for future model integration
shippingservice	Go 1.26	50051	None	GitHub OIDC	Pure calculation; OIDC deploy demo
assistantservice	Python 3.10	8080	None	Gemini API, LangGraph	Python's AI/ML ecosystem; FastAPI async

4.2 Inter-Service Communication Map



4.3 Service API Reference

adservice (Java 25 / Port 9555)

```
GET /ads      Query: context keys → [{redirectUrl, text}]
GET /_healthz → 200 OK
```

authservice (Go / Port 8081)

```
POST /signup      Body: {email, password} → 201 user created
POST /login       Body: {email, password} → 200 {token: "JWT..."}
POST /logout      Header: Authorization → 200 logged out
GET /me           Header: Authorization → 200 {user object}
GET /_healthz     → 200 OK
```

cartservice (.NET / Port 7070)

```
POST /cart/add      Body: {userId, item: {productId, quantity}} → 200
GET /cart/{userId}  → 200 {items: [{productId, quantity}]}
DELETE /cart/{userId} → 204 cart emptied
GET /_healthz       → 200 OK
```

checkoutservice (Go / Port 5050)

```
POST /checkout      Body: {userId, email, address, creditCard} → 200 {orderId}
GET /_healthz       → 200 OK
```

currencyservice (Node.js / Port 7000)

```
GET /currencies      → ["USD", "EUR", "GBP", "INR", ...]
POST /convert        Body: {from, to, amount} → converted amount
GET /_healthz        → 200 OK
```

emailservice (Python / Port 8080)

```
POST /send-confirmation Body: order details → 200 email sent
GET /_healthz          → 200 OK
```

frontend (Go / Ports 8080/80)

```
GET/POST /          Home page (product listing)
GET /product/{id}    Product detail
GET/POST /cart        Cart view / add item
GET/POST /checkout    Checkout / place order
GET/POST /login /signup Auth pages
POST /logout          Log out
POST /setCurrency     Change display currency
GET /_healthz         → "ok"
```

paymentservice (Node.js / Port 50051)

```
POST /charge    Body: {amount:{currencyCode,units,nanos}, creditCard:{...}}  
                → 200 {transactionId: "uuid"} | 400 {error: "validation msg"}  
GET  /_healthz → 200 OK
```

productcatalogservice (Go / Port 3550)

```
GET /products      → [{id, name, description, picture, priceUsd, categories}]  
GET /products/{id} → single product object  
GET /_healthz      → 200 OK
```

shippingservice (Go / Port 50051)

```
GET /getQuote    Query: items → {costUsd: {currencyCode, units, nanos}}  
POST /shipOrder  Body: order → {trackingId: "uuid"}  
GET  /_healthz   → 200 OK
```

assistantservice (Python / Port 8080)

```
POST /api/assistant/chat Body: {productId, message} → {reply: "AI response"}  
GET  /_healthz           → "ok"  
OPTIONS /api/assistant/chat → CORS preflight
```

5. Infrastructure Setup

5.1 Prerequisites Checklist

```
# 1. Kubernetes cluster ready
kubectl cluster-info

# 2. Helm v3 installed
helm version

# 3. ArgoCD in argocd namespace
kubectl get pods -n argocd | grep argocd-server

# 4. NFS StorageClass named "nfs"
kubectl get storageclass nfs

# 5. kGateway controller
kubectl get pods -n kgateway-system

# 6. Sealed Secrets controller
kubectl get pods -n kube-system | grep sealed-secrets

# 7. kubeseal CLI
kubeseal --version

# 8. Argo Rollouts controller (for prod frontend)
kubectl get pods -n argo-rollouts
```

5.2 Complete Bootstrap Sequence

Step 1 — Install ArgoCD

```
kubectl create namespace argocd
kubectl apply -n argocd \
  -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml

# Wait for readiness
kubectl wait --for=condition=Ready pod --all -n argocd --timeout=300s

# Get admin password
kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath="{.data.password}" | base64 -d

# Port-forward to access UI
kubectl port-forward svc/argocd-server -n argocd 8080:443
# https://localhost:8080 | user: admin
```

Step 2 — Install Sealed Secrets

```
helm repo add sealed-secrets https://bitnami-labs.github.io/sealed-secrets
helm repo update
helm install sealed-secrets sealed-secrets/sealed-secrets \
  --namespace kube-system \
  --set fullnameOverride=sealed-secrets-controller

# Verify
kubectl get pods -n kube-system | grep sealed-secrets
```

Step 3 — Install kGateway

```
# Install Kubernetes Gateway API CRDs
kubectl apply -f https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.0.0/standard-install.yaml

# Install kgateway controller (follow official kgateway.dev docs)
helm repo add kgateway https://kgateway.dev/charts
helm install kgateway kgateway/kgateway --namespace kgateway-system --create-namespace
```

Step 4 — Setup NFS StorageClass

Why NFS over hostPath: MongoDB StatefulSet pods may be rescheduled to any node. `hostPath` binds data to one node — if the pod moves, data is gone. NFS provides shared storage accessible from any node.

```
helm repo add nfs-subdir-external-provisioner \
  https://kubernetes-sigs.github.io/nfs-subdir-external-provisioner/

helm install nfs-provisioner \
  nfs-subdir-external-provisioner/nfs-subdir-external-provisioner \
  --set nfs.server=<YOUR-NFS-SERVER-IP> \
  --set nfs.path=/srv/nfs/hipstershop \
  --set storageClass.name=nfs

kubectl get storageclass nfs # Should show: nfs (default provisioner)
```

Step 5 — Install Argo Rollouts

```
kubectl create namespace argo-rollouts
kubectl apply -n argo-rollouts \
  -f https://github.com/argoproj/argo-rollouts/releases/latest/download/install.yaml

# Install kubectl plugin
kubectl argo rollouts version
```

Step 6 — Configure GitHub OIDC (for shippingservice)

Add these flags to kube-apiserver manifest (`/etc/kubernetes/manifests/kube-apiserver.yaml`):

```
- --oidc-issuer-url=https://token.actions.githubusercontent.com
- --oidc-client-id=kubernetes
- --oidc-username-claim=sub
- --oidc-username-prefix=github:
```

Step 7 — Create Namespaces

```
kubectl create namespace hipster-backend --dry-run=client -o yaml | kubectl apply -f -
kubectl create namespace hipster-frontend --dry-run=client -o yaml | kubectl apply -f -
kubectl create namespace hipster-database --dry-run=client -o yaml | kubectl apply -f -
```

Step 8 — Clone Helm Charts Repo and Bootstrap ArgoCD

```
git clone https://github.com/HipsterShopp/hipstershop-helm-charts
cd hipstershop-helm-charts

# Wave 0 – Apply ArgoCD project and infrastructure foundations
kubectl apply -f hipstershop-argoapps/project.yaml
kubectl apply -f hipstershop-argoapps/infra/mongodb.yaml
kubectl apply -f hipstershop-argoapps/infra/backend-common.yaml

# Wait for wave 0 to sync
argocd app get mongodb
argocd app get backend-common

# Wave 1 – Generate and apply Sealed Secrets
cd sealed-secrets/

export JWT_SECRET="your-jwt-secret-here"
export GEMINI_API_KEY="your-gemini-api-key"
export MONGO_ROOT_PASSWORD="your-mongo-root-password"
export AUTH_PASS="your-auth-service-password"
export CART_PASS="your-cart-service-password"
export CATALOG_PASS="your-catalog-service-password"
export ORDER_PASS="your-order-service-password"
export PAYMENT_PASS="your-payment-service-password"
export NOTIFICATION_PASS="your-notification-service-password"
export ANALYTICS_PASS="your-analytics-service-password"

bash generate-sealed-secrets.sh

git add backend-common-secrets.yaml mongodb-secrets.yaml
git commit -m "chore: add sealed secrets"
git push

kubectl apply -f ../hipstershop-argoapps/infra/sealed-secrets.yaml

# Verify secrets decrypted
kubectl get secret app-secrets -n hipster-backend
kubectl get secret mongodb-root -n hipster-backend
kubectl get secret mongodb-users -n hipster-backend
kubectl get secret mongodb-root -n hipster-database
kubectl get secret mongodb-users -n hipster-database

# Wave 2 – Gateway
cd ..
kubectl apply -f hipstershop-argoapps/infra/gateway.yaml
argocd app get gateway # Wait for Synced + Healthy
```

```
# Wave 3 – All Microservices via ApplicationSet
kubectl apply -f hipstershop-argoapps/apps/micro-apps-appset.yaml
kubectl apply -f hipstershop-argoapps/apps/micro-apps-appset-dev.yaml # dev
kubectl apply -f hipstershop-argoapps/apps/micro-apps-appset-prod.yaml # prod

# Watch all apps come online
argocd app list
```

5.3 ArgoCD Sync Wave Order

Wave	Applications	Must Complete Before
0	mongodb , backend-common	Namespaces + ConfigMaps + Secrets ready
1	sealed-secrets	Real Kubernetes Secrets created from SealedSecrets
2	gateway	kGateway + HTTPRoutes accepting traffic
3	All 12 microservices (via ApplicationSets)	Services running and healthy

6. Kubernetes Implementation Deep Dive

6.1 Why Kubernetes

Problem with docker-compose	Kubernetes Solution
No restart on crash	Liveness probes + restart policy
Downtime during deploy	Rolling updates (maxUnavailable: 1)
No horizontal scaling	Replica count + HPA
Manual service discovery	CoreDNS (e.g., authservice:8081)
No resource governance	CPU/Memory requests and limits
Single node	Multi-node scheduling
No secret management	Secrets API + Sealed Secrets

6.2 Full Deployment Manifest (adservice with annotations)

```

# hipstershop-helm-charts/hipstershop-micro-apps/adservice/templates/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  namespace: hipster-backend
  name: adservice
  labels:
    app: adservice
spec:
  replicas: 1 # Single replica – stateless service
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1 # 1 pod can be unavailable during update
      maxSurge: 1 # 1 extra pod allowed during update
  selector:
    matchLabels:
      app: adservice
  template:
    metadata:
      labels:
        app: adservice
        tier: backend
    spec:
      terminationGracePeriodSeconds: 5 # Fast drain – pod has 5s to finish requests
      containers:
        - name: server
          image: yampss/hipstershop-adservice:adservice-v0.1.2 # Injected by CI via Helm
values
  ports:
    - containerPort: 9555
  envFrom:
    - configMapRef:
        name: hipster-config # ENABLE_TRACING, COLLECTOR_SERVICE_ADDR,
GEMINI_MODEL
      env:
        - name: PORT
          valueFrom:
            configMapKeyRef:
              name: service-ports
              key: ADSERVICE_PORT # "9555"
  resources:
    requests:
      cpu: 200m # JVM needs baseline CPU at idle
      memory: 180Mi
    limits:

```

```

    cpu: 300m
    memory: 512Mi                                # JVM heap + OTel agent overhead
readinessProbe:
  initialDelaySeconds: 60                        # JVM + OTel agent takes 50-60s to start
  periodSeconds: 15
  httpGet:
    path: "/_healthz"
    port: 9555
livenessProbe:
  initialDelaySeconds: 60
  periodSeconds: 15
  httpGet:
    path: "/_healthz"
    port: 9555
volumeMounts:
- mountPath: /tmp
  name: tmp-volume                                # OTel agent writes to /tmp
volumes:
- name: tmp-volume
  emptyDir: {}                                    # Ephemeral – cleared on pod restart

```

6.3 MongoDB StatefulSet

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  namespace: hipster-database
  name: mongodb
spec:
  serviceName: mongo-headless          # Required for stable DNS per pod
  replicas: 3                          # 3-node replica set rs0
  selector:
    matchLabels:
      app: mongodb
  template:
    spec:
      containers:
        - name: mongodb
          image: mongo:7.0
          command: ["mongod", "--replSet", "rs0", "--bind_ip_all"]
          ports:
            - containerPort: 27017
          env:
            - name: MONGO_INITDB_ROOT_USERNAME
              valueFrom:
                secretKeyRef:
                  name: mongodb-root
                  key: MONGO_ROOT_USERNAME
            - name: MONGO_INITDB_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mongodb-root
                  key: MONGO_ROOT_PASSWORD
          volumeMounts:
            - name: mongo-data
              mountPath: /data/db
      readinessProbe:
        exec:
          command: ["mongosh", "--eval", "db.adminCommand('ping')"]
          initialDelaySeconds: 20
          periodSeconds: 10
      livenessProbe:
        exec:
          command: ["mongosh", "--eval", "db.adminCommand('ping')"]
          initialDelaySeconds: 30
          periodSeconds: 20
      resources:
        requests:
          cpu: 250m
```

```

        memory: 512Mi
      limits:
        cpu: 500m
        memory: 1Gi
    volumeClaimTemplates:
    - metadata:
        name: mongo-data
      spec:
        accessModes: ["ReadWriteOnce"]
        storageClassName: "nfs"           # Dynamic NFS provisioning
        resources:
          requests:
            storage: 5Gi                 # Creates: mongo-data-mongodb-0/1/2

```

6.4 kGateway Configuration

Why kGateway instead of Ingress:

- Kubernetes Gateway API is the successor to Ingress (more expressive, role-oriented)
- `URLRewrite` filters allow external path mapping without service changes
- Single resource manages routing for all 12 services

```

# GatewayClass (one per cluster)
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: kgateway
spec:
  controllerName: kgateway.dev/kgateway
---
# Gateway (one per platform)
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  namespace: hipster-backend
  name: hipstershop-gateway
spec:
  gatewayClassName: kgateway
  listeners:
  - name: http
    protocol: HTTP
    port: 80
  allowedRoutes:
    namespaces:
      from: All      # Accepts HTTPRoutes from all namespaces (including hipster-frontend)

```

6.5 HTTPRoute Routing Table

External Path	Rewrites To	Backend Service	Port
GET /api/ads	/ads	adservice	9555
POST /api/auth/login	/login	authservice	8081
POST /api/auth/signup	/signup	authservice	8081
POST /api/auth/logout	/logout	authservice	8081
GET /api/auth/me	/me	authservice	8081
PathPrefix /api/cart	/cart/*	cartservice	7070
POST /api/checkout	/checkout	checkoutservice	5050
GET /api/currency/currencies	/currencies	currencyservice	7000
POST /api/currency/convert	/convert	currencyservice	7000
POST /api/email/send-confirmation	/send-confirmation	emailservice	8080
POST /api/payment/charge	/charge	paymentservice	50051
GET /api/products/*	/products/*	productcatalogservice	3550
GET /api/recommendations	/recommendations	recommendationservice	8080
PathPrefix /api/shipping	/	shippingservice	50051
POST /api/assistant/chat	(no rewrite)	assistantservice	8080
/, /product, /cart, /login, /signup, /static/*, /_healthz	(direct)	frontend	80

6.6 Network Policies (Zero-Trust)

```

# Policy 1: default-deny-all (hipster-backend)
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: hipster-backend
spec:
  podSelector: {}          # All pods
  policyTypes:
    - Ingress
    - Egress
---
# Policy 2: backend-allow (controlled exceptions)
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-allow
  namespace: hipster-backend
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
  ingress:
    - from:
      - ipBlock:
          cidr: 172.31.0.0/16          # VPC CIDR (cluster nodes)
    - from:
      - namespaceSelector:
          matchLabels:
            name: hipster-frontend    # Allow frontend → backend
    - from:
      - podSelector: {}              # Allow intra-backend communication
  egress:
    - to:
      - namespaceSelector:
          matchLabels:
            name: hipster-database    # Backend → MongoDB
    - to:
      - podSelector: {}              # Intra-backend pods
    - to:
      - namespaceSelector:
          matchLabels:
            name: hipster-frontend
    - to:
      - namespaceSelector:
          matchLabels:
            kubernetes.io/metadata.name: kgateway-system

```

```

- to:
  - namespaceSelector: {}
  ports:
  - protocol: UDP
    port: 53 # DNS resolution
  - protocol: TCP
    port: 53
---
# Policy 3: emailservice SMTP egress only
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: emailservice-allow-smtp
  namespace: hipster-backend
spec:
  podSelector:
    matchLabels:
      app: emailservice
  policyTypes: [Egress]
  egress:
  - to:
    - ipBlock:
        cidr: 0.0.0.0/0 # Gmail SMTP (any IP)
    ports:
    - protocol: TCP
      port: 587 # SMTP with STARTTLS

```

6.7 RBAC for GitHub OIDC (ShippingService)

```
# hipstershop-infra/rbac/github-actions-oidc-shippingservice.yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: github-actions-shippingservice-verifier
  namespace: hipster-backend
rules:
- apiGroups: ["apps"]
  resources: ["deployments"]
  verbs: ["get", "list", "patch", "update"] # kubectl set image
- apiGroups: ["apps"]
  resources: ["replicasets"]
  verbs: ["get", "list"] # rollout status
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list"] # verify pod state
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: github-actions-shippingservice-verifier
  namespace: hipster-backend
subjects:
- kind: User
  # OIDC subject: exact repo + exact branch – develop branch automatically denied
  name: "github:repo:HipsterShopp/hipstershop-shippingservice:ref:refs/heads/main"
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: github-actions-shippingservice-verifier
  apiGroup: rbac.authorization.k8s.io
```

6.8 HPA — Frontend Autoscaling

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: frontend
  namespace: hipster-frontend
spec:
  scaleTargetRef:
    apiVersion: argoproj.io/v1alpha1
    kind: Rollout
    name: frontend
  minReplicas: 1
  maxReplicas: 5
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 80
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 20    # Conservative scale-down
      policies:
        - type: Percent
          value: 50
          periodSeconds: 15
    scaleUp:
      stabilizationWindowSeconds: 5    # Aggressive scale-up for traffic spikes
      policies:
        - type: Percent
          value: 100
          periodSeconds: 15
        - type: Pods
          value: 2
          periodSeconds: 15

```

7. Helm-Based Deployment Strategy

7.1 Why Helm

Before Helm, each service needed manually written Kubernetes YAMLs. With 12 services each needing a Deployment + Service (minimum), that is 24+ files — all with repeated boilerplate. Values like namespace names and port numbers were copy-pasted, creating configuration drift. Helm eliminates this by centralizing all configuration and using template injection.

Why Helm in this project specifically:

- 12 services × 3 environments (base/dev/prod) = managed with just 3 values files per service
- ArgoCD uses Helm's template engine — `helm template` output is applied via `kubectl`
- Versioned releases with rollback capability
- One `sed` command in CI can update the image tag in values.yaml → ArgoCD picks it up

7.2 Repository Structure

```

hipstershop-helm-charts/
├─ hipstershop-argoapps/           # ArgoCD Application/ApplicationSet manifests
│  └─ project.yaml                 # ArgoCD AppProject: hipstershop
│     └─ apps/
│        │ └─ micro-apps-appset.yaml # Main: 11 services, values.yaml
│        │ └─ micro-apps-appset-dev.yaml # Dev: 12 services, values.yaml + values.dev.yaml
│        │ └─ micro-apps-appset-prod.yaml # Prod: 11 services, values.yaml + values.prod.yaml
│        └─ infra/
│           │ └─ mongodb.yaml         # Wave 0
│           │ └─ backend-common.yaml  # Wave 0
│           │ └─ sealed-secrets.yaml  # Wave 1
│           │ └─ gateway.yaml         # Wave 2
│           └─ loadgenerator.yaml     # Wave 3
├─ hipstershop-infra/              # Infrastructure Helm charts
│  │ └─ backend-common/             # Namespace, ConfigMaps, Secrets, NetworkPolicy
│  │ └─ gateway/                   # kGateway GatewayClass + Gateway + HTTPRoutes
│  │ └─ mongodb/                   # MongoDB StatefulSet, Services, Init Job
│  └─ rbac/                        # GitHub OIDC RBAC for shipping-service
├─ hipstershop-micro-apps/         # One Helm chart per microservice
│  │ └─ adservice/
│  │    │ └─ Chart.yaml
│  │    │ └─ values.yaml           # Base (main branch defaults)
│  │    │ └─ values.dev.yaml       # Dev overrides (lighter resources, develop image)
│  │    │ └─ values.prod.yaml      # Prod overrides (GHCR image, full resources)
│  │    └─ templates/
│  │       │ └─ deployment.yaml
│  │       └─ service.yaml
│  └─ [authservice, cartservice, checkoutservice, currencyservice,
│      emailservice, frontend, paymentservice, productcatalogservice,
│      recommendationservice, shipping-service, assistant-service]
│      └─ (same structure)
└─ sealed-secrets/
   │ └─ backend-common-secrets.yaml # Encrypted app-secrets (safe to git commit)
   │ └─ mongodb-secrets.yaml       # Encrypted MongoDB credentials
   │ └─ generate-sealed-secrets.sh  # Generator script
   └─ steps.txt                    # Operator runbook

```

7.3 Three-Layer Values Override System

File	Used In	Purpose
values.yaml	Base/Fallback	Default config, base image tags

values.dev.yaml	Dev ApplicationSet	Lighter resources (50%), develop image tag
values.prod.yaml	Prod ApplicationSet	Full resources, GHCR image tags

ArgoCD renders via:

```
helm template {service} hipstershop-micro-apps/{service} \
-f hipstershop-micro-apps/{service}/values.yaml \
-f hipstershop-micro-apps/{service}/values.dev.yaml    # or values.prod.yaml
```

7.4 Values Comparison (adservice)

```
# values.yaml (base)
global:
  images:
    adservice: "yampss/hipstershop-adservice:adservice-v0.1.2"
  replicas:
    adservice: 1
  resources:
    adservice:
      requests:
        cpu: 200m
        memory: 180Mi
      limits:
        cpu: 300m
        memory: 512Mi
  ports:
    adservice: 9555

# values.dev.yaml (dev overrides – lighter resources)
global:
  images:
    adservice: "yampss/hipstershop-adservice:develop-latest" # Updated by CI
  resources:
    adservice:
      requests:
        cpu: 100m # 50% of prod
        memory: 90Mi # 50% of prod
      limits:
        cpu: 200m
        memory: 256Mi

# values.prod.yaml (prod overrides – GHCR + full resources)
global:
  images:
    adservice: "ghcr.io/hipstershopp/hipstershop-adservice:adservice-v0.2.3"
  resources:
    adservice:
      requests:
        cpu: 200m
        memory: 180Mi
      limits:
        cpu: 300m
        memory: 512Mi
```

7.5 backend-common ConfigMaps

5 ConfigMaps in `hipster-backend` (created by Wave 0):

ConfigMap	Keys	Purpose
hipster-config	ENABLE_TRACING, COLLECTOR_SERVICE_ADDR, FRONTEND_MESSAGE, ENABLE_ASSISTANT	App-wide feature flags
service-addresses	PRODUCT_CATALOG_SERVICE_ADDR, SHIPPING_SERVICE_ADDR, PAYMENT_SERVICE_ADDR, EMAIL_SERVICE_ADDR, CURRENCY_SERVICE_ADDR, CART_SERVICE_ADDR, GATEWAY_ADDR	Inter-service DNS addresses
service-ports	ADSERVICE_PORT=9555, AUTHSERVICE_PORT=8081, CARTSERVICE_PORT=7070, CHECKOUTSERVICE_PORT=5050, CURRENCYSERVICE_PORT=7000, EMAILSERVICE_PORT=8080, PAYMENTSERVICE_PORT=50051, PRODUCTCATALOG_PORT=3550, SHIPPINGSERVICE_PORT=50051	All service ports
mongodb-config	MONGO_REPLICA_SET=rs0, AUTH_DB_NAME, CART_DB_NAME, CATALOG_DB_NAME, PAYMENT_DB_NAME, PAYMENT_CHARGES_COLLECTION	MongoDB database/collection names
app-features	CURRENCY_SERVICE_RANDOM_ERROR=1, PAYMENT_SERVICE_DISABLE_PROFILER=1, PRODUCT_CATALOG_SEED_DATABASE=true, GEMINI_MODEL=gemini-2.5-flash	Feature flags

7.6 Helm Commands Reference

```
# NOTE: helm list returns empty – ArgoCD uses helm template + kubectl apply, not helm install
helm list -n hipster-backend    # Returns empty – EXPECTED

# Render templates locally (debugging)
helm template adservice hipstershop-micro-apps/adservice \
  -f hipstershop-micro-apps/adservice/values.yaml

# Render with dev overrides
helm template adservice hipstershop-micro-apps/adservice \
  -f hipstershop-micro-apps/adservice/values.yaml \
  -f hipstershop-micro-apps/adservice/values.dev.yaml

# Lint chart
helm lint hipstershop-micro-apps/adservice \
  -f hipstershop-micro-apps/adservice/values.yaml

# Dry run
helm install adservice hipstershop-micro-apps/adservice \
  -f hipstershop-micro-apps/adservice/values.yaml \
  --dry-run
```

8. CI/CD Pipeline

8.1 Design Philosophy

Goal: Automate the full loop — code push → quality gate → build → scan → push → deploy.

Two environments:

- `develop` — continuous deployment, auto-synced by ArgoCD (no human gate)
- `main` — **manual approval gate** before production deployment

Why shared workflows (not per-repo pipelines):

Before `hipstershop-shared-workflows`, every service repo duplicated 200+ lines of CI YAML. Updating the Trivy action version required 12 separate PRs. With shared workflows, updating a tool version = **one PR** to `hipstershop-shared-workflows`, all 12 services pick it up automatically.

8.2 Shared Workflow Catalog

Workflow	Purpose	Triggered On
<code>sonar-and-snyk.yaml</code>	SonarQube Quality Gate + Snyk HIGH+ dep scan	PR to main/develop
<code>generate-tag-and-build-image-trivy.yaml</code>	Semver/branch tag, docker build, Trivy CRITICAL scan	Push + PR
<code>push.yaml</code>	Push to DockerHub (:versioned + :latest), create git tag	Push to main/develop
<code>helm-updater-job.yaml</code>	Checkout helm-charts repo, <code>sed</code> tag in values file, commit+push	After push
<code>notify.yaml</code>	SendGrid failure email with job status dashboard	Always (on failure)

All callers reference `@feature/loose-coupling` branch:

```
uses: HipsterShopp/hipstershop-shared-workflows/.github/workflows/sonar-and-snyk.yaml@feature/loose-coupling
```

8.3 Full Pipeline Flow

push to develop or main (src/** changed)

[PR only] sonar-and-snyk

SonarQube Quality Gate

Snyk: snyk test --severity-threshold=high

Upload Snyk HTML report (14-day retention)

(skip on push; always run on PR success)

generate-tag-and-build-image-trivy

main: mathieudutour/github-tag-action → adservice-v0.1.3

develop: develop-{sha8} (e.g., develop-3f2a1c9b)

Check if tag already exists on DockerHub

docker build -t {user}/hipstershop-{service}:{tag} src/

docker save → /tmp/image.tar (artifact)

[PR] Trivy: severity=CRITICAL, ignore-unfixed=true

(push event only)

push

docker load -i image.tar

docker login DockerHub

docker push :versioned-tag

docker push :latest (main only)

git tag {tag} && git push (main only)

develop

main

update-helm-values-dev

checkout helm-charts:develop

sed tag in values.dev.yaml

git commit + push

push-to-ghcr

ghcr.io/hipstershopp/{service}:{tag}

prod-approval-gate

GitHub Environment: production 🙌

update-helm-values-prod

checkout helm-charts:main

sed tag in values.prod.yaml

git commit + push

[shippingservice only]

deploy-to-prod (self-hosted runner)

GitHub OIDC → kubectl set image

ArgoCD detects diff (~3 min)

micro-apps-appset-dev

ArgoCD detects diff (~3 min)

micro-apps-appset-prod

auto-sync → rolling update auto-sync → rolling update / blue-green

[Always] notify → SendGrid email on any failure

8.4 Tag Strategy

```
# develop branch → branch-slug + short SHA
BRANCH_SLUG=$(echo "$GITHUB_REF_NAME" | sed 's|/|-|g')
SHORT_SHA=$(echo "$GITHUB_SHA" | cut -c1-8)
TAG="${BRANCH_SLUG}-${SHORT_SHA}"
# Example: develop-3f2a1c9b

# main branch → semantic version with service prefix
# Uses mathieudutour/github-tag-action@v6.2
# tag_prefix: "{service_name}-v"
# Example: adservice-v0.1.3
```

8.5 Docker Build Strategy

All services use **multi-stage builds** to minimize runtime image size:

Service	Builder Image	Runtime Image	Why Runtime Choice
adservice	eclipse-temurin:24-jdk	eclipse-temurin:25-jre-alpine	JRE only (no compiler) at runtime
authservice	golang:1.26-alpine	gcr.io/distroless/static-debian12	No OS = zero CVE surface
cartservice	dotnet/sdk:8.0	dotnet/aspnet:8.0	ASP.NET runtime only
checkoutservice	golang:1.26-alpine	gcr.io/distroless/static	Distroless Go
currencyservice	node:20-alpine	alpine:3.23 + nodejs	Minimal alpine + node
emailservice	python:3.11-alpine	python:3.11-alpine	Python Alpine
frontend	golang:1.26-alpine	gcr.io/distroless/static	Distroless + templates/static dirs
paymentservice	node:20-alpine	alpine:3.23 + nodejs	Same as currencyservice
productcatalogservice	golang:1.26-alpine	gcr.io/distroless/static	Distroless Go
recommendationservice	python:3.11-alpine	python:3.11-alpine	Python Alpine
shippingservice	golang:1.26-alpine	gcr.io/distroless/static	Distroless Go
assistantservice	—	python:3.10-slim	Single stage (no native compilation)

Key build flags:

- `CGO_ENABLED=0` — All Go services: produces static binary, required for distroless
- `-ldflags="-s -w"` — Strips debug symbols, reduces binary size
- `--only=production` — Node.js: exclude devDependencies from runtime image
- `sed -i 's/\r$//' gradlew` — adservice: fix Windows CRLF line endings on Linux CI runners

8.6 Required Secrets per Service Repo

Secret	Purpose	Set At
<code>DOCKERHUB_USERNAME</code>	Docker Hub login	Repo secrets
<code>DOCKERHUB_TOKEN</code>	Docker Hub push	Repo secrets
<code>GHCR_TOKEN</code>	GHCR push (prod images)	Repo secrets
<code>HELM_CHARTS_TOKEN</code>	Push to helm-charts repo	Repo secrets
<code>SNYK_TOKEN</code>	Snyk scan authentication	Repo secrets
<code>SONAR_TOKEN</code>	SonarQube authentication	Repo secrets
<code>SONAR_HOST_URL</code>	SonarQube server URL	Repo secrets
<code>SENDGRID_API_KEY</code>	Failure email notifications	Repo secrets
<code>K8S_API_SERVER</code>	shippingservice only — cluster API URL	Repo secrets
<code>K8S_CA_CERT</code>	shippingservice only — cluster CA (base64)	Repo secrets

8.7 Thin Caller Pattern (adservice example)

```

# hipstershop-adservice/.github/workflows/adservice.yaml
name: adservice CI

on:
  push:
    branches: [main, develop]
    paths: ['src/**', '.github/workflows/adservice.yaml']
  pull_request:
    branches: [main, develop]
    paths: ['src/**', '.github/workflows/adservice.yaml']

permissions:
  contents: write
  packages: write

jobs:
  sonar-and-snyk:
    if: github.event_name == 'pull_request'
    uses: HipsterShopp/hipstershop-shared-workflows/.github/workflows/sonar-and-snyk.yaml@feature/loose-coupling
    with:
      service_name: adservice
      service_path: src/
      runtime: java
      java-version: "21"
      java-build-tool: "gradle"
    secrets:
      SNYK_TOKEN: ${ secrets.SNYK_TOKEN }
      SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
      SONAR_HOST_URL: ${ secrets.SONAR_HOST_URL }

  generate-tag-and-build-image:
    needs: [sonar-and-snyk]
    if: |
      always() &&
      (
        (github.event_name == 'pull_request' && needs.sonar-and-snyk.result == 'success')
      ||
        github.event_name == 'push'
      )
    uses: HipsterShopp/hipstershop-shared-workflows/.github/workflows/generate-tag-and-build-image-trivy.yaml@feature/loose-coupling
    with:
      service_name: adservice
      service_path: src/
      run_trivy: ${ github.event_name == 'pull_request' }

```

```
secrets:
  DOCKERHUB_USERNAME: ${ secrets.DOCKERHUB_USERNAME }
  DOCKERHUB_TOKEN: ${ secrets.DOCKERHUB_TOKEN }

# ... push, push-to-ghcr, prod-approval-gate, update-helm-values-prod, notify
```

8.8 helm-updater-job — The GitOps Trigger

This is the critical link between CI and ArgoCD:

```
# Checkout hipstershop-helm-charts repo (not the service repo)
git checkout hipstershop-helm-charts:develop

# Update ONLY the tag line in the values file
sed -i "s|^(\s*tag:\s*\).*|\1${IMAGE_TAG}|" \
  hipstershop-micro-apps/adservice/values.dev.yaml

# Commit (idempotent – skips if no change)
git diff --cached --quiet && echo "No changes" && exit 0
git commit -m "chore(adservice): update image tag to ${IMAGE_TAG}"
git push origin develop
```

ArgoCD polls this repo every ~3 minutes. When it detects this commit, it triggers sync.

8.9 Pipeline Timing

Stage	Typical Duration
CI checks on PR (sonar + build + trivy)	6–12 minutes
Docker build + push (push event)	3–8 minutes
ArgoCD detection (git poll)	up to 3 minutes
Pod start + readiness (Java)	60–90 seconds
Pod start + readiness (Go/Python/Node)	5–15 seconds
Total end-to-end (push → running pod)	~10–15 minutes

9. GitOps Deployment — ArgoCD

9.1 Why GitOps

Problem with push-based CD (GitHub Actions running `kubectl apply`):

- If someone manually changes a Kubernetes resource (`kubectl edit deployment/adservice`), the change is invisible
- No audit trail of what's running vs what's in git

- No automatic recovery from cluster drift

ArgoCD GitOps solution:

- Git is the **only** source of truth
- ArgoCD continuously reconciles cluster state against git (`selfHeal: true`)
- Drift detection: any manual change is automatically reverted
- One-click rollback to any previous git state

9.2 ArgoCD AppProject

```
apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: hipstershop
  namespace: argocd
spec:
  description: "HipsterShop microservices platform"
  sourceRepos:
    - "https://github.com/HipsterShopp/hipstershop-helm-charts"
  destinations:
    - server: https://kubernetes.default.svc
      namespace: hipster-backend
    - server: https://kubernetes.default.svc
      namespace: hipster-frontend
    - server: https://kubernetes.default.svc
      namespace: hipster-database
    - server: https://kubernetes.default.svc
      namespace: kube-system
  clusterResourceWhitelist:
    - group: "*"
      kind: "*"
  orphanedResources:
    warn: true
```

9.3 ApplicationSet — Scaling Solution

Instead of 12 individual Application YAMLs, one ApplicationSet generates all of them:

```
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: micro-apps-dev
  namespace: argocd
spec:
  generators:
    - list:
        elements:
          - name: adservice
            namespace: hipster-backend
          - name: assistantservice
            namespace: hipster-backend
          - name: authservice
            namespace: hipster-backend
          - name: cartservice
            namespace: hipster-backend
          - name: checkoutservice
            namespace: hipster-backend
          - name: currencyservice
            namespace: hipster-backend
          - name: emailservice
            namespace: hipster-backend
          - name: frontend
            namespace: hipster-frontend      # Only service NOT in hipster-backend
          - name: paymentservice
            namespace: hipster-backend
          - name: productcatalogservice
            namespace: hipster-backend
          - name: recommendationservice
            namespace: hipster-backend
          - name: shippingservice
            namespace: hipster-backend

  template:
    metadata:
      name: "{{name}}-dev"
      namespace: argocd
    annotations:
      argocd.argoproj.io/sync-wave: "3"
    spec:
      project: hipstershop
      source:
        repoURL: https://github.com/HipsterShopp/hipstershop-helm-charts
        targetRevision: develop
        path: "hipstershop-micro-apps/{{name}}"
```

```

helm:
  valueFiles:
    - values.yaml
    - values.dev.yaml
destination:
  server: https://kubernetes.default.svc
  namespace: "{{namespace}}"
syncPolicy:
  automated:
    selfHeal: true
    prune: true
  syncOptions:
    - CreateNamespace=false
    - ServerSideApply=true

```

Adding a new microservice: Add one element to `generators.list.elements` + create `hipstershop-micro-apps/{new-service}/` Helm chart. ArgoCD picks it up on next sync.

9.4 Sync Policies by Application

Application	Auto-sync	selfHeal	prune	Reason
mongodb	Yes	Yes	No	Never auto-delete database resources
backend-common	Yes	Yes	No	ConfigMaps should not be auto-deleted
gateway	Yes	Yes	Yes	Stateless config — safe to prune
micro-apps (dev)	Yes	Yes	Yes	Dev: always converge fast
micro-apps (prod)	Yes	Yes	No	Prod: safer, leave orphaned resources

9.5 Sync Wave Order

```

Wave 0:  mongodb + backend-common
        ↓ (Synced + Healthy)
Wave 1:  sealed-secrets
        ↓ (Real Secrets decrypted in cluster)
Wave 2:  gateway
        ↓ (HTTPRoutes active)
Wave 3:  All 12 microservices via ApplicationSet
        ↓ (Services running and serving traffic)
Wave 4:  loadgenerator (disabled by default)

```

9.6 ArgoCD Commands

```
# Login
argocd login localhost:8080 --username admin --password <password> --insecure

# List all apps
argocd app list

# Get app status (sync status, health, current image)
argocd app get adservice-dev

# Force sync
argocd app sync adservice-dev

# Force refresh (re-poll git without syncing)
argocd app get adservice-dev --refresh

# View rendered manifests
argocd app manifests adservice-dev

# View diff (git vs cluster)
argocd app diff adservice-dev

# View history
argocd app history adservice-dev

# Rollback to previous version
argocd app rollback adservice-dev <history-id>

# Promote frontend blue-green (prod)
kubectl argo rollouts promote frontend -n hipster-frontend
```

9.7 Important: helm list Returns Empty

```
$ helm list -n hipster-backend
NAME    NAMESPACE    REVISION    STATUS    CHART    APP VERSION
(empty)
```

This is EXPECTED. ArgoCD uses `helm template + kubectl apply` — it never runs `helm install`. No Helm release is registered in cluster history. Use `argocd app list` instead.

10. Deployment Strategies

10.1 Rolling Update (All Services Except Frontend Prod)

Default strategy for all 12 microservices:

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 1    # 1 pod can be down – minimum availability
    maxSurge: 1          # 1 extra pod during update – avoids capacity issues
```

Flow:

1. New pod starts alongside old pod
2. New pod passes readiness probe
3. Old pod receives `SIGTERM`, waits `terminationGracePeriodSeconds: 5`
4. Old pod terminated — zero downtime (new pod already serving)

10.2 Blue-Green Deployment (Frontend Production)

Why Blue-Green for frontend: The frontend serves HTML templates and static assets that change visually. A rolling update mixing old and new frontend pods during promotion would serve inconsistent UI. Blue-Green ensures a clean cutover with the ability to preview before promoting.

```
# Argo Rollouts Rollout (replaces Deployment in prod)
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: frontend
  namespace: hipster-frontend
spec:
  strategy:
    blueGreen:
      activeService: frontend-active    # Routes live user traffic
      previewService: frontend-preview # Staging preview (no live traffic)
      autoPromotionEnabled: false      # Human must approve promotion
      scaleDownDelaySeconds: 60        # Keep Blue alive 60s after promotion
```

Blue-Green Flow:

```
New image tag in values.prod.yaml
|
▼
ArgoCD syncs Rollout spec
|
▼
Argo Rollouts creates GREEN ReplicaSet
|
▼
Green pods pass readiness probe → attached to frontend-preview service
|
▼
🛑 PAUSED – operator reviews preview URL
|
▼
kubectl argo rollouts promote frontend -n hipster-frontend
|
▼
frontend-active service selector flips → GREEN pods serve live traffic
|
▼
Blue RS kept 60s (rollback window) → scaled to 0
```

Quick rollback:

```
# One command reverts to previous known-good version
kubectl argo rollouts undo frontend -n hipster-frontend
```

10.3 GitHub OIDC Deploy (ShippingService Production)

ShippingService skips ArgoCD entirely in production. A self-hosted runner authenticates with GitHub OIDC and runs `kubectl set image` directly:

```

deploy-to-prod:
  runs-on: self-hosted
  permissions:
    id-token: write    # Required to request OIDC token
    contents: read
  steps:
  - name: Request GitHub OIDC token
    run: |
      TOKEN=$(curl -sSf \
        -H "Authorization: bearer $ACTIONS_ID_TOKEN_REQUEST_TOKEN" \
        "${ACTIONS_ID_TOKEN_REQUEST_URL}&audience=kubernetes" \
        | jq -r '.value')
      echo "token=$TOKEN" >> $GITHUB_OUTPUT

  - name: Configure kubectl (OIDC — no stored kubeconfig)
    run: |
      echo "${{ secrets.K8S_CA_CERT }}" | base64 -d > ca.crt
      kubectl config set-cluster prod-cluster \
        --server=${{ secrets.K8S_API_SERVER }} \
        --certificate-authority=ca.crt --embed-certs=true
      kubectl config set-credentials github-actions-oidc \
        --token="${{ steps.oidc.outputs.token }}"
      kubectl config set-context prod \
        --cluster=prod-cluster --user=github-actions-oidc
      kubectl config use-context prod

  - name: Deploy new image
    run: |
      IMAGE="yampss/hipstershop-shippingservice:${TAG}"
      kubectl set image deployment/shippingservice server="${IMAGE}" -n hipster-backend
      kubectl rollout status deployment/shippingservice -n hipster-backend --timeout=5m

```

Security properties:

- No `kubeconfig` stored in GitHub Secrets — only cluster URL and CA cert (non-sensitive)
- JWT expires in ~5 minutes after issuance
- RBAC RoleBinding locks to exact repo + exact branch (`refs/heads/main`)
- `develop` branch automatically denied (different OIDC `sub` claim)

11. Security Architecture

11.1 Defense in Depth

Layer 1: Network Policies	– default-deny-all; explicit allow rules only
Layer 2: Sealed Secrets	– all secrets encrypted in Git; cluster-specific decryption
Layer 3: RBAC	– least-privilege roles for GitHub OIDC
Layer 4: Container Security	– distroless (no shell); non-root (cartservice: USER 1000)
Layer 5: Image Scanning	– Trivy CRITICAL CVE scan on every PR build
Layer 6: Dependency Scanning	– Snyk HIGH+ vulnerability scan on every PR
Layer 7: Transport Security	– inter-service via ClusterIP (cluster-internal network)
Layer 8: Authentication	– JWT (authservice); OIDC (shippingservice prod deploy)
Layer 9: Git Security	– Branch protection; PR required; human approval for prod

11.2 Sealed Secrets

```
plaintext Secret
|
▼ kubeseal --cert cluster.pem
SealedSecret (safe to git commit)
|
▼ Sealed Secrets controller (kube-system) decrypts with cluster private key
Kubernetes Secret (in cluster only)
```

Secrets sealed:

- `app-secrets` — `JWT_SECRET`, `GEMINI_API_KEY`
- `mongodb-root` — root admin credentials
- `mongodb-users` — per-service MongoDB credentials (auth, cart, catalog, order, payment, notification, analytics)

Re-sealing after cluster rebuild:

```
cd hipstershop-helm-charts/sealed-secrets/

# Re-fetch cluster public cert
kubeseal --fetch-cert \
  --controller-name=sealed-secrets-controller \
  --controller-namespace=kube-system > pub-cert.pem

# Re-export all secrets and re-seal
export JWT_SECRET="..."
# ... all other exports
bash generate-sealed-secrets.sh

git add backend-common-secrets.yaml mongodb-secrets.yaml
git commit -m "chore: reseal secrets for new cluster"
git push
```

11.3 Container Security

Practice	Services	Detail
Distroless runtime	All Go services + some	No shell, no package manager, minimal CVE surface
Non-root user	cartservice	<code>USER 1000</code> in Dockerfile
SHA-pinned base images	Go, Node, Java services	Reproducible builds; upstream tag changes don't break builds
No hardcoded secrets	All services	Secrets from Kubernetes Secrets mounted as env vars
Multi-stage builds	All except assistantservice	Runtime image contains only compiled artifact

11.4 TLS / Transport

- All inter-service traffic is within the cluster (ClusterIP to ClusterIP) — Kubernetes cluster network
- External traffic enters via kGateway on port 80 (HTTP)
- **Production recommendation:** TLS termination at kGateway level (cert-manager + Let's Encrypt)
- MongoDB connections use SCRAM-SHA-256 authentication

11.5 JWT Authentication

```

User POST /api/auth/login
  |
  ▼
authservice validates credentials → issues JWT signed with JWT_SECRET
  |
  ▼
Frontend stores JWT in cookie
  |
  ▼
Subsequent requests carry JWT → frontend validates with JWT_SECRET
(authservice handles revocation via /logout)

```

12. Observability

12.1 Current State

Component	Status	Tool	Coverage
Distributed Tracing	✓ Implemented	Jaeger (OpenTelemetry)	adservice (Java OTel agent), Go services (OTel SDK)
Structured Logging	✓ Implemented	Logrus (JSON stdout)	authservice; all services log to stdout

Metrics	⚠️ Planned	Prometheus (not deployed)	Gap — no alerting or dashboards
Log Aggregation	⚠️ Planned	Loki/ELK (not deployed)	Gap — logs accessible via <code>kubectl logs</code> only

12.2 Distributed Tracing (Jaeger)

All services that support OpenTelemetry connect to:

```
COLLECTOR_SERVICE_ADDR: "jaeger:4317"
ENABLE_TRACING: "1"
```

adservice — bakes OTEL Java agent into Docker image:

```
RUN wget -O /opentelemetry-javaagent.jar \
  https://github.com/open-telemetry/opentelemetry-java-
instrumentation/releases/latest/download/opentelemetry-javaagent.jar
ENV JAVA_TOOL_OPTIONS="-javaagent:/opentelemetry-javaagent.jar"
```

12.3 Structured Logging

authservice uses logrus for JSON-structured logs:

```
log.WithFields(log.Fields{
  "method":    r.Method,
  "path":      r.URL.Path,
  "user_email": email,
}).Info("login attempt")
```

Output:

```
{"level":"info","method":"POST","path":"/login","time":"2026-04-
22T10:00:00Z","user_email":"user@test.com","msg":"login attempt"}
```

12.4 Viewing Logs

```
# Real-time logs
kubectl logs -f <pod-name> -n hipster-backend

# Logs from crashed container
kubectl logs <pod-name> -n hipster-backend --previous

# Logs by label (pod name changes with each deploy)
kubectl logs -l app=adservice -n hipster-backend --tail=100

# Logs from all replicas simultaneously
kubectl logs -l app=mongodb -n hipster-database --prefix=true

# Resource usage
kubectl top pods -n hipster-backend
kubectl top nodes
```

12.5 Recommended Future Observability Stack

Prometheus + Grafana (kube-prometheus-stack)

- CPU, Memory, Request Rate, Error Rate, Latency per service
- Alertmanager for threshold-based alerts (PagerDuty/Slack)

Grafana Loki + Promtail DaemonSet

- Centralized log aggregation
- Correlation IDs across services
- 30-day log retention

Jaeger (already configured)

- Request traces across all 12 services
- Latency breakdown per service hop

13. Performance & Scalability

13.1 Resource Allocation

Service	CPU Request	CPU Limit	Memory Request	Memory Limit	Reason
adservice	200m	300m	180Mi	512Mi	JVM heap + OTel agent overhead
authservice	100m	200m	64Mi	128Mi	Go binary — minimal baseline
cartservice	100m	200m	64Mi	256Mi	.NET runtime overhead

checkoutservice	100m	200m	64Mi	128Mi	Go binary — orchestration only
currencyservice	50m	100m	32Mi	64Mi	Node.js — data transformation only
emailservice	50m	100m	64Mi	128Mi	Python — infrequent calls
frontend	100m	300m	64Mi	256Mi	Go binary + template rendering
paymentservice	50m	100m	64Mi	128Mi	Node.js
productcatalogservice	100m	200m	64Mi	128Mi	Go binary + MongoDB queries
recommendationservice	50m	100m	64Mi	128Mi	Python — read-only
shippingservice	50m	100m	32Mi	64Mi	Go — pure calculation
assistantservice	100m	300m	128Mi	512Mi	Python + AI inference calls
MongoDB (per pod)	250m	500m	512Mi	1Gi	Database engine

Dev values use ~50% of above requests (resource savings on dev cluster).

13.2 Scalability Approach

- **Frontend HPA** — scales 1→5 replicas based on CPU (70%) and Memory (80%)
- **Stateless services** — all 11 backend services are stateless (state in MongoDB); any can scale horizontally
- **MongoDB replica set** — 3 nodes provide read scaling; primary handles writes
- **Connection pooling** — Go MongoDB driver uses connection pool; Node.js services pool connections

14. Networking Architecture

14.1 Traffic Flows

```

External User
  | HTTP :80
  ▼
kGateway LoadBalancer (cluster external IP)
  | path matching (11 HTTPRoute objects)
  | URLRewrite filter
  ▼
ClusterIP Service (hipster-backend or hipster-frontend namespace)
  | kube-proxy DNAT
  ▼
Pod (container port)
  | (response)
  ▼
Back through gateway → User

```

14.2 Internal DNS

Kubernetes CoreDNS resolves service names:

```

authservice.hipster-backend.svc.cluster.local:8081
cartservice.hipster-backend.svc.cluster.local:7070
mongodb-0.mongo-headless.hipster-database.svc.cluster.local:27017

```

Services reference each other using short names configured in ConfigMaps:

```

# service-addresses ConfigMap
CART_SERVICE_ADDR: "cartservice:7070"
EMAIL_SERVICE_ADDR: "emailservice:8080"
GATEWAY_ADDR: "hipstershop-gateway.hipster-backend.svc.cluster.local:80"

```

14.3 Namespace Isolation

```

hipster-frontend  —————→ hipster-backend  (via NetworkPolicy allow)
hipster-backend  —————→ hipster-database (via NetworkPolicy allow)
hipster-backend  —x————— internet          (blocked by NetworkPolicy, except SMTP)
hipster-database —x————— hipster-frontend (no policy allows this)

```

15. Configuration Management

15.1 ConfigMap Strategy

All non-sensitive runtime configuration is stored in ConfigMaps in `hipster-backend` :

```
# hipster-config – app-wide behavior
ENABLE_TRACING: "1"
COLLECTOR_SERVICE_ADDR: "jaeger:4317"
FRONTEND_MESSAGE: "Checkout HipsterShop today!"
ENABLE_ASSISTANT: "true"

# service-addresses – inter-service DNS
PRODUCT_CATALOG_SERVICE_ADDR: "productcatalogservice:3550"
CART_SERVICE_ADDR: "cartservice:7070"
GATEWAY_ADDR: "hipstershop-gateway.hipster-backend.svc.cluster.local:80"

# mongodb-config – DB/collection names
MONGO_REPLICA_SET: "rs0"
AUTH_DB_NAME: "auth_db"
CART_DB_NAME: "cart_db"
PAYMENT_DB_NAME: "payment_db"
PAYMENT_CHARGES_COLLECTION: "charges"

# app-features – feature flags
CURRENCY_SERVICE_RANDOM_ERROR: "1"           # Chaos simulation
PRODUCT_CATALOG_SEED_DATABASE: "true"        # Auto-seed products
GEMINI_MODEL: "gemini-2.5-flash"
```

15.2 Secret Management

Secret	Location	Contents	Encrypted
app-secrets	hipster-backend	JWT_SECRET, GEMINI_API_KEY	Via Sealed Secrets
mongodb-root	hipster-backend + hipster-database	MONGO_ROOT_USERNAME/PASSWORD	Via Sealed Secrets
mongodb-users	hipster-backend + hipster-database	Per-service user/password pairs	Via Sealed Secrets

15.3 Environment Variables Injection

```
# In deployment template – multiple injection sources
containers:
- name: server
  envFrom:
  - configMapRef:
      name: hipster-config          # Bulk non-sensitive config
  - configMapRef:
      name: service-addresses      # Inter-service DNS
  env:
  - name: PORT
    valueFrom:
      configMapKeyRef:
        name: service-ports
        key: AUTHSERVICE_PORT
  - name: JWT_SECRET
    valueFrom:
      secretKeyRef:
        name: app-secrets
        key: JWT_SECRET
  - name: MONGO_USERNAME
    valueFrom:
      secretKeyRef:
        name: mongodb-users
        key: AUTH_MONGO_USERNAME
```

16. 12-Factor App Compliance

Factor	Implementation
I — Codebase	Each service in its own Git repo; one codebase, multiple deploys (dev/prod)
II — Dependencies	Explicitly declared (go.mod, package.json, requirements.txt, build.gradle, .csproj)
III — Config	All config in env vars from ConfigMaps/Secrets — no hardcoded values
IV — Backing Services	MongoDB treated as attached resource via URI env var — swap without code change
V — Build/Release/Run	CI builds Docker image artifact → Helm values release → ArgoCD deploys
VI — Processes	All containers are stateless; state exclusively in MongoDB StatefulSet
VII — Port Binding	Each service exports HTTP via PORT env var; no shared server
VIII — Concurrency	Scale via Kubernetes replica count + HPA
IX — Disposability	terminationGracePeriodSeconds: 5 ; fast startup (distroless Go); health probes

X — Dev/Prod Parity	Same Docker images; same Helm charts; different values.dev vs values.prod
XI — Logs	All services log to stdout (<code>kubectl logs</code>); structured JSON (authservice)
XII — Admin Processes	MongoDB init jobs run as Kubernetes Jobs; seed jobs as one-off Pods

17. Disaster Recovery & Backup

17.1 Current State

Data	Location	Backup	RPO	RTO
MongoDB data	NFS PVCs (5Gi × 3)	✗ No automated backup	Undefined	Undefined
Git state (configs)	GitHub repos	✓ Git history	Near-zero	Minutes
Docker images	DockerHub + GHCR	✓ Registry redundancy	Near-zero	Minutes
Helm charts	GitHub + ArgoCD	✓ Git history	Near-zero	Minutes
Sealed Secrets	Git (encrypted)	✓ Git + private key safe	Near-zero	Minutes

17.2 MongoDB Backup Strategy (Recommended)

```

# CronJob: Daily MongoDB backup to S3
apiVersion: batch/v1
kind: CronJob
metadata:
  name: mongodb-backup
  namespace: hipster-database
spec:
  schedule: "0 2 * * *"          # 2 AM daily
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: backup
              image: mongo:7.0
              command:
                - sh
                - -c
                - |
                  mongodump \
                    --host mongodb-0.mongo-headless.hipster-database.svc.cluster.local:27017 \
                    --username $MONGO_ROOT_USERNAME \
                    --password $MONGO_ROOT_PASSWORD \
                    --authenticationDatabase admin \
                    --out /backup/$(date +%Y%m%d)
                  # Upload to S3 or NFS backup location
              envFrom:
                - secretRef:
                    name: mongodb-root
          restartPolicy: OnFailure

```

17.3 Restore Procedure

```
# 1. Restore from mongodump
kubectl exec -it mongodb-0 -n hipster-database -- bash
mongorestore \
  --host mongodb-0.mongo-headless.hipster-database.svc.cluster.local:27017 \
  --username admin --password <root-password> \
  --authenticationDatabase admin \
  /backup/20260422/

# 2. Rollback ArgoCD to previous version
argocd app history adservice-prod
argocd app rollback adservice-prod <history-id>

# 3. Rollback frontend blue-green
kubectl argo rollouts undo frontend -n hipster-frontend

# 4. Rollback Kubernetes deployment directly
kubectl rollout undo deployment/adservice -n hipster-backend
kubectl rollout history deployment/adservice -n hipster-backend
```

17.4 Cluster Rebuild Checklist

```
# 1. Re-install ArgoCD
# 2. Re-install Sealed Secrets controller
# 3. Re-fetch cluster public cert for re-sealing
kubeseal --fetch-cert --controller-name=sealed-secrets-controller \
  --controller-namespace=kube-system > pub-cert.pem

# 4. Re-seal secrets with new cluster key
bash generate-sealed-secrets.sh
git add backend-common-secrets.yaml mongodb-secrets.yaml
git commit -m "chore: reseal for new cluster" && git push

# 5. Re-apply ArgoCD bootstrap sequence (waves 0→3)
# 6. Re-configure GitHub OIDC in kube-apiserver flags
# 7. Re-add GitHub secrets: K8S_API_SERVER, K8S_CA_CERT
```

18. Runbook — Operations Guide

18.1 Standard Deploy Flow

```
# Dev (automatic after PR merge to develop)
# 1. Developer merges PR to develop
# 2. GitHub Actions: build → push DockerHub → update values.dev.yaml
# 3. ArgoCD: auto-detects → auto-syncs → rolling update
# Verify:
argocd app get <service>-dev
kubectl rollout status deployment/<service> -n hipster-backend

# Prod (requires manual approval)
# 1. Developer merges PR to main
# 2. GitHub Actions: build → push GHCR → PAUSE for approval
# 3. Team lead approves GitHub Environment "production"
# 4. Actions: update values.prod.yaml
# 5. ArgoCD: auto-syncs → rolling update (or blue-green for frontend)
# Verify:
argocd app get <service>-prod
kubectl get deployment <service> -n hipster-backend \
  -o jsonpath='{.spec.template.spec.containers[0].image}'
```

18.2 Frontend Blue-Green Promotion

```
# 1. CI pushes new image tag → values.prod.yaml updated → ArgoCD syncs Rollout
# 2. Argo Rollouts creates GREEN ReplicaSet
# 3. Green pods pass readiness → attached to frontend-preview service

# Check rollout status
kubectl argo rollouts get rollout frontend -n hipster-frontend --watch

# Preview new version (access frontend-preview service)
kubectl port-forward svc/frontend-preview 8080:80 -n hipster-frontend
curl http://localhost:8080/_healthz

# Promote GREEN to live traffic
kubectl argo rollouts promote frontend -n hipster-frontend

# Emergency abort (if green has issues)
kubectl argo rollouts abort frontend -n hipster-frontend

# Rollback (revert to blue)
kubectl argo rollouts undo frontend -n hipster-frontend
```

18.3 Emergency Rollback Procedures

```
# Option 1: ArgoCD rollback (preferred – reverts git state)
argocd app history adservice-dev
argocd app rollback adservice-dev <revision-id>

# Option 2: Kubernetes rollback (immediate – doesn't update git)
kubectl rollout undo deployment/adservice -n hipster-backend
kubectl rollout status deployment/adservice -n hipster-backend

# Option 3: Manual image tag update
# Edit hipstershop-micro-apps/adservice/values.dev.yaml
# Change image tag to last known good
git commit -m "revert: rollback adservice to adservice-v0.1.1"
git push # ArgoCD picks up and syncs

# Option 4: shippingservice rollback (OIDC deploy)
# Must run new deployment manually or trigger CI again
kubectl set image deployment/shippingservice \
  server=yampss/hipstershop-shippingservice:<previous-tag> \
  -n hipster-backend
```

18.4 Scaling Operations

```
# Manual scale (temporary – ArgoCD will revert to values.yaml replica count)
kubectl scale deployment/adservice --replicas=3 -n hipster-backend

# Permanent scale – update values.yaml in helm-charts and push
# Edit: hipstershop-micro-apps/adservice/values.yaml
# global.replicas.adservice: 3
git commit -m "scale: increase adservice replicas to 3"
git push # ArgoCD syncs

# Restart a deployment (e.g., to pick up ConfigMap changes)
kubectl rollout restart deployment/adservice -n hipster-backend

# Force ArgoCD re-sync
argocd app sync adservice-dev --force
```

18.5 Secret Rotation

```
# 1. Generate new secret value
NEW_JWT_SECRET=$(openssl rand -base64 32)

# 2. Re-seal
cd hipstershop-helm-charts/sealed-secrets/
export JWT_SECRET="$NEW_JWT_SECRET"
# ... export other secrets unchanged
bash generate-sealed-secrets.sh

# 3. Commit and push new sealed secrets
git add backend-common-secrets.yaml
git commit -m "chore: rotate JWT_SECRET"
git push

# 4. Also update GitHub Secrets if needed (for CI workflows)
# Go to each service repo → Settings → Secrets → Update

# 5. ArgoCD will sync new SealedSecret → new Kubernetes Secret
# 6. Bounce authservice to pick up new JWT_SECRET
kubectl rollout restart deployment/authservice -n hipster-backend
```

19. Troubleshooting Guide

19.1 Pod States Reference

State	Meaning	First Action
Pending	Pod can't be scheduled	<code>kubectl describe pod <name> -n hipster-backend</code> — check Events
ImagePullBackOff	Can't pull image from registry	Check image tag exists on DockerHub/GHCR
CrashLoopBackOff	Container starts then crashes	<code>kubectl logs <pod> -n hipster-backend --previous</code>
OOMKilled	Out of memory	Increase memory limit in values.yaml
Running 0/1	Pod running but not Ready	Readiness probe failing — check logs + <code>/_healthz</code> endpoint

19.2 Issue: CrashLoopBackOff

```
# Step 1: Get crash reason
kubectl logs <pod-name> -n hipster-backend --previous

# Step 2: Check exit code
kubectl describe pod <pod-name> -n hipster-backend | grep -A3 "Last State"

# Step 3: Check events
kubectl get events -n hipster-backend --sort-by='.lastTimestamp' | tail -20

# Common causes:
# authservice: "Required environment variable MONGO_URI is not set"
kubectl get secret app-secrets -n hipster-backend
kubectl get secret mongodb-users -n hipster-backend

# adservice: OOM killed – increase memory limit
kubectl top pod -l app=adservice -n hipster-backend

# paymentservice: MongoDB refused – service continues but check logs
kubectl logs -l app=payment-service -n hipster-backend | head -20
```

19.3 Issue: ImagePullBackOff

```
kubectl describe pod <pod-name> -n hipster-backend | grep -A5 "Events:"
# "Failed to pull image ..."

# Check if image exists on DockerHub
docker pull yampss/hipstershop-adservice:develop-abc12345

# Check ArgoCD rendered the right image
argocd app manifests adservice-dev | grep image:

# Check the actual deployment
kubectl get deployment adservice -n hipster-backend \
  -o jsonpath='{.spec.template.spec.containers[0].image}'
```

19.4 Issue: ArgoCD OutOfSync (Never Heals)

```
# Check what is different
argocd app diff adservice-dev

# Common cause: controller modifies fields ArgoCD keeps reverting
# For gateway – add ignoreDifferences:
ignoreDifferences:
- group: gateway.networking.k8s.io
  kind: GatewayClass
  jsonPointers: [/metadata/annotations, /metadata/labels, /spec/parametersRef]

# For frontend blue-green – add ignoreDifferences on prod AppSet:
ignoreDifferences:
- group: ""
  kind: Service
  name: frontend-active
  namespace: hipster-frontend
  jsonPointers: [/spec/selector]
```

19.5 Issue: Service Unreachable (502/503 via Gateway)

```
# Step 1: Check if pod is Running and Ready
kubectl get pods -n hipster-backend -l app=<service>

# Step 2: Check if Service has Endpoints (pods registered)
kubectl get endpoints <service> -n hipster-backend
# Should show: ADDRESSES not empty

# Step 3: Check HTTPRoute status
kubectl get httproute -n hipster-backend
kubectl describe httproute <route-name> -n hipster-backend

# Step 4: Check NetworkPolicy (is traffic blocked?)
kubectl describe networkpolicy -n hipster-backend

# Step 5: Test from inside cluster
kubectl exec -it <any-running-pod> -n hipster-backend -- \
  curl http://<service>:<port>/_healthz
```

19.6 Issue: MongoDB Pod Pending

```
kubectl describe pod mongodb-0 -n hipster-database
# Look for: "no persistent volumes available for this claim"
# OR: "node had taints that the pod didn't tolerate"

# Check PVC status
kubectl get pvc -n hipster-database

# Check if NFS StorageClass exists
kubectl get storageclass nfs

# Check NFS provisioner
kubectl get pods -n default | grep nfs    # or wherever provisioner runs

# Fix if StorageClass missing: re-install NFS provisioner (see section 5.4)
```

19.7 Issue: Sealed Secret Not Decrypting

```
# Check controller is running
kubectl get pods -n kube-system | grep sealed-secrets

# Check for events on the SealedSecret
kubectl describe sealedsecret app-secrets -n hipster-backend

# Check controller logs
kubectl logs -l app.kubernetes.io/name=sealed-secrets -n kube-system --tail=50

# Common cause: sealed with different cluster key (after cluster rebuild)
# Fix: re-seal all secrets with new cluster public key (see section 17.4)
```

19.8 Issue: GitHub Actions Fails on Docker Push

```
# Error: unauthorized: authentication required
# 1. Check DOCKERHUB_TOKEN is set in repo secrets
# 2. Token must have Read & Write scope
# 3. Regenerate at hub.docker.com → Account Settings → Security → New Access Token
# 4. Update GitHub secret in repo Settings → Secrets and variables → Actions
```

19.9 Issue: helm-updater-job Fails

```
# Error: Permission denied
# HELM_CHARTS_TOKEN must have write access to hipstershop-helm-charts repo
# Scope: repo (full access) or at minimum: contents:write
# Regenerate at GitHub → Settings → Developer settings → Personal access tokens

# Error: Nothing to commit
# Image tag is the same (CI ran but image unchanged)
# This is harmless – sed found no change, git exit 0
```

19.10 Issue: authservice Crashes on Startup

```
kubectl logs -l app=authservice -n hipster-backend --previous
# "Required environment variable MONGO_URI is not set" → secrets missing
# "Required environment variable JWT_SECRET is not set" → secrets missing

# Verify sealed secrets were applied and decrypted
kubectl get secret app-secrets -n hipster-backend
kubectl get secret mongodb-users -n hipster-backend

# If secrets missing, apply sealed secrets file
kubectl apply -f hipstershop-helm-charts/sealed-secrets/backend-common-secrets.yaml

# Verify ArgoCD sealed-secrets app is Synced
argocd app get sealed-secrets
```

20. Challenges & Solutions

#	Challenge	Root Cause	Solution
1	CrashLoopBackOff on adservice	OOM killed — JVM heap exceeded 256Mi limit	Increased memory limit to 512Mi to accommodate JVM + OTel agent
2	exec format error in adservice CI	gradlew committed with Windows CRLF; Linux can't execute CRLF scripts	Added <code>sed -i 's/\r\$//' gradlew && chmod +x gradlew</code> to Dockerfile
3	no such file or directory in distroless Go images	CGO was enabled → dynamically linked binary; distroless has no libc	Added <code>CGO_ENABLED=0</code> to all Go build commands
4	ArgoCD perpetually OutOfSync for gateway	kGateway controller mutates GatewayClass annotations; ArgoCD keeps reverting	Added <code>ignoreDifferences</code> for GatewayClass annotations/labels

5	helm list returns empty — "nothing deployed"	ArgoCD uses helm template + kubectl apply , not helm install	Documented expected behavior; use argocd app list instead
6	ShippingService prod deploy — "Unauthorized"	OIDC JWT sub claim didn't match RoleBinding subject exactly	Corrected subject to: github:repo:HipsterShopp/hipstershop-shippingservice:ref:refs/heads/main
7	MongoDB pod Pending — no PVC bound	NFS StorageClass not provisioned before StatefulSet deployment	Pre-provision NFS StorageClass as part of cluster bootstrap (wave 0 prerequisite)
8	ARM race: frontend ArgoCD + Argo Rollouts fighting over Service selector	Rollouts controller modifies Service selector during promotion; ArgoCD reverts	Added ignoreDifferences for frontend-active/frontend-preview Service selectors
9	MongoDB authservice init-container timing	authservice tries to connect before MongoDB primary is elected	Added init-container that polls MongoDB until a writable primary is available
10	PR CI running for non-src changes	Workflow triggered on any push, rebuilding even when only README changed	Added paths: ['src/**'] filter to GitHub Actions triggers
11	12 repos × 200-line CI files = maintenance nightmare	No shared CI library — every repo had its own full pipeline YAML	Created hipstershop-shared-workflows with 5 reusable workflows; callers are 30-100 lines
12	Sealed Secrets incompatible after cluster rebuild	Sealed Secrets are encrypted with cluster-specific RSA key pair	Documented re-sealing process; generate-sealed-secrets.sh automates it
13	currencyservice random errors confusing checkout debugging	CURRENCY_SERVICE_RANDOM_ERROR: "1" is intentional chaos	Added documentation — this is intentional; set to "0" for debugging

21. Architecture Decision Records

ADR-001: Kubernetes over Docker Swarm / VMs

Decision: Use Kubernetes for container orchestration.

Reason:

- Industry standard for production microservices
- Self-healing via liveness probes and restart policies
- Rolling updates without downtime

- HPA for CPU/memory-based autoscaling
- Native support for StatefulSets (required for MongoDB)
- Rich ecosystem: ArgoCD, Sealed Secrets, Argo Rollouts all require Kubernetes

Alternative Considered: Docker Swarm — simpler but lacks StatefulSets, HPA, Gateway API, and the ecosystem tools used here.

Trade-off: Significantly higher operational complexity than Docker Swarm. New engineers require Kubernetes training.

ADR-002: Helm for Kubernetes Manifest Templating

Decision: Use Helm v3 as the Kubernetes manifest templating system.

Reason:

- Eliminates 24+ manually written YAML files (12 services × Deployment + Service)
- Centralizes all configuration in `values.yaml`
- Three-layer override system (base / dev / prod) maps cleanly to environments
- ArgoCD has native Helm support — renders `helm template` output without `helm install`
- Versioned chart releases support rollbacks

Alternative Considered: Kustomize — more declarative but less powerful for multi-environment value management.

Trade-off: Helm's templating language (Go templates) has a learning curve. `helm list` returning empty confuses engineers unfamiliar with ArgoCD + Helm integration.

ADR-003: ArgoCD with ApplicationSet for GitOps

Decision: Use ArgoCD with ApplicationSet-driven deployment instead of Flux or push-based CD.

Reason:

- Declarative GitOps — git is the only source of truth
- `selfHeal: true` automatically reverts any manual cluster changes
- ApplicationSet generates all 12 Application manifests from one template
- Sync waves ensure correct deployment order (database → config → gateway → services)
- Web UI provides visual sync status for all services

Alternative Considered: Flux v2 — similar capabilities but less mature web UI; ArgoCD team had more experience with it.

Trade-off: ArgoCD adds operational complexity (separate server to maintain). `helm list` shows no releases — confuses engineers used to direct Helm deployment.

ADR-004: Shared GitHub Actions Workflows

Decision: Create `hipstershop-shared-workflows` repository with 5 shared workflows. All 12 service repos call these shared workflows via `uses:` reference.

Reason:

- Without shared workflows: 12 repos × 200-line YAML = 2,400 lines of duplicated CI config
- Any tool version update (Trivy, SonarQube scanner) requires 12 PRs
- With shared workflows: one PR to shared repo propagates to all 12 services

Trade-off: Pipeline debugging requires looking at two repos (service repo for trigger, shared-workflows for logic). Adds coupling on `feature/loose-coupling` branch name.

ADR-005: GitHub OIDC for ShippingService Production Deploy

Decision: ShippingService production deployment uses GitHub OIDC JWT authentication directly to kubectl — not through ArgoCD.

Reason:

- Demonstrates zero-credential production deployment pattern
- No stored kubeconfig or long-lived tokens in GitHub Secrets
- JWT expires in ~5 minutes — minimal attack window
- RBAC RoleBinding locked to exact repo + exact branch

Trade-off: ShippingService ArgoCD Application may show `OutOfSync` after production deploy (running image differs from `values.prod.yaml`). Hybrid approach adds complexity.

ADR-006: Sealed Secrets for Git-Safe Secret Management

Decision: Use Bitnami Sealed Secrets to encrypt Kubernetes Secrets before committing to git.

Reason:

- Cannot store plain Kubernetes Secret manifests in git (base64 is not encryption)
- Sealed Secrets encrypts with cluster RSA public key — only the specific cluster can decrypt
- Safe to commit encrypted SealedSecret objects to public or private repos
- Simple workflow: `kubeseal --cert cluster.pem < secret.yaml > sealed-secret.yaml`

Alternative Considered: External Secrets Operator with AWS Secrets Manager — more powerful (rotation, audit trail) but significantly more complex setup.

Trade-off: Cluster-specific — sealed secrets must be re-generated if cluster RSA key rotates or cluster is rebuilt.

ADR-007: Multi-Repo Architecture (14 repos vs 1 monorepo)

Decision: 14 separate repositories — one per microservice, plus shared-workflows, plus helm-charts.

Reason:

- Monorepo with 12 services: any code change triggers CI for ALL services simultaneously

- Independent repos: CI only runs when that service's `src/**` changes
- Separate repos enable per-service access control (payment team doesn't need frontend access)
- ArgoCD monitors `hipstershop-helm-charts` only — not individual service repos

Trade-off: More repos to manage. Cross-service changes require coordinating multiple PRs. Developer onboarding requires cloning multiple repos.

ADR-008: MongoDB Replica Set (3 nodes) over Single Instance

Decision: Deploy MongoDB as a 3-node replica set (`rs0`) using `StatefulSet`.

Reason:

- Single MongoDB instance = single point of failure for all services
- Replica set provides automatic failover when primary fails
- Read scaling — replica nodes can serve read queries
- Services authenticate with SCRAM-SHA-256 using per-service dedicated databases and credentials

Trade-off: 3× the resource consumption. NFS storage required (not `hostPath`). MongoDB replica set initialization complexity (init job required).

22. Environment Strategy

22.1 Three-Environment Model

	Dev	Prod
Branch	<code>develop</code>	<code>main</code>
Image registry	DockerHub (<code>yampss/</code>)	DockerHub + GHCR (<code>ghcr.io/hipstershopp/</code>)
Image tag format	<code>develop-{sha8}</code>	<code>service-vX.Y.Z</code>
Resource requests	50% of prod	Full
Argo Rollouts	No (Deployment)	Yes (frontend Blue-Green)
ArgoCD prune	Yes	No
Human approval gate	No	Yes
shippingservice deploy	ArgoCD (<code>values.dev.yaml</code>)	GitHub OIDC <code>kubectl</code>

22.2 Promoting from Dev to Prod

```
# 1. Verify dev environment is healthy
argocd app list    # All services Synced + Healthy on dev cluster
kubectl get pods -n hipster-backend    # All Running

# 2. Create PR from develop → main
# GitHub Actions: sonar + snyk + build + trivy + push DockerHub

# 3. Team lead reviews and approves PR
# GitHub Actions: push-to-ghcr → PAUSE at prod-approval-gate

# 4. Team lead approves GitHub Environment "production"
# GitHub Actions: update-helm-values-prod → ArgoCD syncs prod cluster

# 5. Verify prod
argocd app list --server <prod-argocd-server>
kubectl get pods -n hipster-backend    # All Running on prod cluster
```

23. Release & Versioning Strategy

23.1 Semantic Versioning

All services follow semantic versioning with service-name prefix:

- **Format:** {service-name}-vMAJOR.MINOR.PATCH
- **Example:** adservice-v0.1.3, frontend-v1.2.0

Version is calculated automatically from conventional commit messages:

- **feat:** → MINOR bump (v0.1.x → v0.2.0)
- **fix:** → PATCH bump (v0.1.2 → v0.1.3)
- **feat!:** or **BREAKING CHANGE:** → MAJOR bump

23.2 Release Lifecycle

```
feature/* branch (developer)
  | PR to develop
  ▼
develop branch → CI build → DockerHub (develop-{sha8}) → Dev cluster
  | PR to main (tech lead review)
  ▼
main branch → CI build → DockerHub + GHCR (service-vX.Y.Z) → Prod approval
  | human approval
  ▼
Production cluster updated
  | git tag created automatically
  ▼
argocd app history shows versioned release (rollback-able)
```

23.3 Git Tag Conventions

```
# Service-specific tags (created automatically by push.yaml on main)
adservice-v0.1.3
frontend-v1.2.0
authservice-v0.3.1

# View all tags for a service
git tag --list "adservice-*" | sort -V

# Rollback to previous tag
argocd app history adservice-prod
argocd app rollback adservice-prod <revision>
```

24. Incident Management & SLA

24.1 Severity Levels

Severity	Description	Example	Response Time	Resolution Target
P1 — Critical	Full platform outage	Gateway down, all services unreachable	≤ 15 minutes	≤ 2 hours
P2 — High	Core feature unavailable	Checkout broken, MongoDB primary down	≤ 30 minutes	≤ 4 hours
P3 — Medium	Non-critical service degraded	emailservice down, recommendations unavailable	≤ 2 hours	≤ 24 hours
P4 — Low	Minor issues	Slow AI assistant, currency random errors	Next business day	≤ 72 hours

24.2 Incident Response Runbook

```
# P1: Full Platform Outage

# 1. Check gateway
kubectl get pods -n kgateway-system
kubectl get gateway hipstershop-gateway -n hipster-backend -o yaml

# 2. Check all pods
kubectl get pods -n hipster-backend
kubectl get pods -n hipster-frontend
kubectl get pods -n hipster-database

# 3. Check ArgoCD sync status
argocd app list

# 4. Check recent deployments (was a bad deploy the cause?)
kubectl rollout history deployment/frontend -n hipster-frontend
argocd app history frontend-prod

# 5. If bad deploy: rollback
argocd app rollback <app-name> <revision>
# OR:
kubectl rollout undo deployment/<service> -n hipster-backend

# 6. Check node health
kubectl get nodes
kubectl describe node <node-name> | grep -A10 Conditions

# P2: MongoDB Primary Down
kubectl get pods -n hipster-database
kubectl exec -it mongodb-0 -n hipster-database -- \
  mongosh --eval "rs.status()" --username admin --password <root-pw> --
authenticationDatabase admin
# Should show PRIMARY + 2 SECONDARYs
# If no primary: force re-election
kubectl exec -it mongodb-0 -n hipster-database -- \
  mongosh --eval "rs.reconfig({...}, {force: true})"
```

24.3 Alerting (Current / Recommended)

Current: SendGrid email on CI pipeline failure only (via `notify.yaml` shared workflow).

Recommended additional alerts:

```
# Prometheus AlertManager rules (to implement)
- alert: PodCrashLooping
  expr: rate(kube_pod_container_status_restarts_total[5m]) > 0
  for: 5m
  annotations:
    summary: "Pod {{ $labels.pod }} is crash looping"

- alert: MongoDBReplicaSetDegraded
  expr: mongodb_rs_members_health{state!="1"} < 2
  for: 2m
  annotations:
    summary: "MongoDB replica set has < 2 healthy members"

- alert: HighErrorRate
  expr: rate(http_requests_total{status=~"5.."}[5m]) > 0.05
  for: 5m
  annotations:
    summary: "Error rate > 5% on {{ $labels.service }}"
```

25. Non-Functional Requirements

NFR	Target	Current Implementation	Status
Availability	≥ 99.9%	Rolling updates + MongoDB replica set + ArgoCD self-healing	✅ Design supports
Scalability	HPA 1→5× frontend	HPA configured (CPU 70%, Memory 80%)	✅ Implemented
Security	RBAC + OIDC + Sealed Secrets	Network policies + Sealed Secrets + RBAC + distroless	✅ Implemented
Performance	< 500ms p99 API response	Distroless Go services (fast); Java 60s startup	⚠️ Java needs probe tuning
Reliability	Self-healing on probe failure	Liveness + readiness probes on all 12 services	✅ Implemented
Auditability	Git history for all changes	All changes via git; ArgoCD audit log; CI logs	✅ Implemented
Observability	Full request tracing	OpenTelemetry → Jaeger (partial); kubectl logs	⚠️ Prometheus/Grafana pending
DR	RTO < 1 hour	ArgoCD re-sync restores services; MongoDB backup pending	⚠️ Backup not automated

Data Isolation	Per-service databases	7+ isolated MongoDB databases with per-service credentials	✅ Implemented
----------------	-----------------------	--	---------------

26. Cost & Resource Optimization

26.1 Current Optimizations

Optimization	Example	Savings
Multi-stage Docker builds	All services — runtime image 60-80% smaller than build image	Faster pulls, lower registry storage
Distroless runtime images	All Go services — ~10MB runtime image	Faster pulls, minimal CVEs
Dev resource halving	CPU 200m→100m, Memory 180Mi→90Mi in values.dev.yaml	~50% resource savings on dev cluster
Layer caching	go.mod/package.json copied before src — deps cached	Faster CI builds on unchanged deps
Single gateway (vs per-service LB)	kGateway replaces 12 LoadBalancer services	11 fewer cloud load balancers
ApplicationSet (vs 12 Apps)	One ArgoCD ApplicationSet generates all 12 apps	Operational simplicity

26.2 Recommended Optimizations

- **Enable HPA for backend services** — productcatalogservice and checkoutservice can benefit from autoscaling during peak traffic
- **Implement Pod Disruption Budgets** — prevent accidental full-service outage during node maintenance
- **Add Pod Anti-Affinity** — spread replicas across nodes for true HA
- **Consider multi-stage builds for assistantservice** — currently single-stage; add builder for pip dependency compilation
- **MongoDB connection pooling audit** — verify all services use connection pooling efficiently

27. Future Enhancements

Enhancement	Current Gap	Recommended Solution	Effort	Impact
Prometheus + Grafana	No metrics/alerting	kube-prometheus-stack Helm chart + ServiceMonitors	Medium	High
Grafana Loki	kubectl logs only	Loki + Promtail DaemonSet	Medium	High
External Secrets Operator	Sealed Secrets (manual re-seal)	ESO + AWS Secrets Manager / HashiCorp Vault	High	High

HPA for backend services	Frontend HPA only	Add HPA for productcatalogservice, checkoutservice	Low	Medium
Pod Anti-Affinity	Single-node risk	<code>podAntiAffinity</code> in Helm templates	Low	Medium
Pod Disruption Budgets	No maintenance protection	PDB for MongoDB and frontend	Low	Medium
TLS at Gateway	HTTP only	cert-manager + Let's Encrypt + TLS listener in kGateway	Medium	High
Trivy config scan	Image scan only	Add <code>trivy config</code> scan on Helm templates in CI	Low	Medium
MongoDB automated backup	Manual only	Velero + MongoDB CronJob → S3	Medium	High
Istio service mesh	No mTLS	Istio sidecar injection + mTLS policies	High	High
Cost optimization	Fixed resources	KEDA + custom metrics autoscaling	High	Medium
Canary deployments	Blue-green only	Argo Rollouts canary strategy for backend services	Medium	Medium

28. Appendix — Full Command Reference

28.1 Kubernetes Commands

— PODS —

```
kubectl get pods -n hipster-backend
kubectl get pods -n hipster-frontend
kubectl get pods -n hipster-database
kubectl get pods -n argocd

kubectl describe pod <pod-name> -n hipster-backend
kubectl logs <pod-name> -n hipster-backend
kubectl logs <pod-name> -n hipster-backend --previous      # crashed container
kubectl logs -f <pod-name> -n hipster-backend              # follow
kubectl logs -l app=adservice -n hipster-backend --tail=100 # by label
```

— DEPLOYMENTS —

```
kubectl get deployments -n hipster-backend
kubectl rollout status deployment/adservice -n hipster-backend
kubectl rollout restart deployment/adservice -n hipster-backend
kubectl rollout undo deployment/adservice -n hipster-backend
kubectl rollout history deployment/adservice -n hipster-backend
```

— SERVICES & NETWORKING —

```
kubectl get svc -n hipster-backend
kubectl get endpoints adservice -n hipster-backend
kubectl get networkpolicy -n hipster-backend
kubectl get httproute -n hipster-backend
kubectl get gateway -n hipster-backend
```

— PORT-FORWARDING (for local testing) —

```
kubectl port-forward svc/adservice 9555:9555 -n hipster-backend
kubectl port-forward svc/authservice 8081:8081 -n hipster-backend
kubectl port-forward svc/cartservice 7070:7070 -n hipster-backend
kubectl port-forward svc/checkoutservice 5050:5050 -n hipster-backend
kubectl port-forward svc/currencyservice 7000:7000 -n hipster-backend
kubectl port-forward svc/emailservice 8080:8080 -n hipster-backend
kubectl port-forward svc/frontend 8080:80 -n hipster-frontend
kubectl port-forward svc/paymentservice 50051:50051 -n hipster-backend
kubectl port-forward svc/productcatalogservice 3550:3550 -n hipster-backend
kubectl port-forward svc/shippingservice 50051:50051 -n hipster-backend
kubectl port-forward svc/assistant-service 8080:8080 -n hipster-backend
kubectl port-forward svc/argocd-server 8080:443 -n argocd
```

— RESOURCES & DEBUG —

```
kubectl top pods -n hipster-backend
kubectl top nodes
kubectl get events -n hipster-backend --sort-by='.lastTimestamp'
kubectl get events -n hipster-database --sort-by='.lastTimestamp'
```

```
# — SECRETS & CONFIGMAPS —
```

```
kubectl get secret app-secrets -n hipster-backend
kubectl get secret mongodb-root -n hipster-backend
kubectl get secret mongodb-users -n hipster-backend
kubectl get secret mongodb-root -n hipster-database
kubectl get secret mongodb-users -n hipster-database

kubectl get configmap hipster-config -n hipster-backend -o yaml
kubectl get configmap service-addresses -n hipster-backend -o yaml
kubectl get configmap service-ports -n hipster-backend -o yaml
kubectl get configmap mongodb-config -n hipster-backend -o yaml
```

```
# — MONGODB —
```

```
kubectl get pods -n hipster-database -l app=mongodb
kubectl get statefulset mongodb -n hipster-database
kubectl get pvc -n hipster-database

kubectl exec -it mongodb-0 -n hipster-database -- mongosh \
  --username admin --password <root-password> --authenticationDatabase admin

# Check replica set status
kubectl exec -it mongodb-0 -n hipster-database -- \
  mongosh --eval "rs.status()" --username admin --password <pw> --authenticationDatabase
admin
```

```
# — STORAGE —
```

```
kubectl get storageclass
kubectl get pv
kubectl get pvc -n hipster-database
kubectl describe pvc mongo-data-mongodb-0 -n hipster-database
```

28.2 ArgoCD Commands

```
# Login
argocd login localhost:8080 --username admin --password <password> --insecure

# List all applications
argocd app list

# Get detailed app status
argocd app get adservice-dev
argocd app get frontend-prod

# Sync an application
argocd app sync adservice-dev
argocd app sync adservice-dev --force

# Force refresh (re-poll git)
argocd app get adservice-dev --refresh

# View rendered manifests (what ArgoCD applies)
argocd app manifests adservice-dev

# View diff (git state vs cluster state)
argocd app diff adservice-dev

# View history
argocd app history adservice-dev

# Rollback to previous revision
argocd app rollback adservice-dev <revision-id>
```

28.3 Helm Commands

```
# IMPORTANT: helm list returns empty – ArgoCD manages releases
helm list -n hipster-backend    # Always empty – EXPECTED

# From hipstershop-helm-charts directory:

# Render templates locally
helm template adservice hipstershop-micro-apps/adservice \
  -f hipstershop-micro-apps/adservice/values.yaml

# Render with dev overrides
helm template adservice hipstershop-micro-apps/adservice \
  -f hipstershop-micro-apps/adservice/values.yaml \
  -f hipstershop-micro-apps/adservice/values.dev.yaml

# Render with prod overrides
helm template adservice hipstershop-micro-apps/adservice \
  -f hipstershop-micro-apps/adservice/values.yaml \
  -f hipstershop-micro-apps/adservice/values.prod.yaml

# Lint chart
helm lint hipstershop-micro-apps/adservice \
  -f hipstershop-micro-apps/adservice/values.yaml

# Dry run (shows what would be applied without ArgoCD)
helm install adservice hipstershop-micro-apps/adservice \
  -f hipstershop-micro-apps/adservice/values.yaml \
  --dry-run --debug
```

28.4 Argo Rollouts Commands (Frontend)

```
# Check rollout status
kubectl argo rollouts get rollout frontend -n hipster-frontend
kubectl argo rollouts get rollout frontend -n hipster-frontend --watch

# Promote GREEN to live traffic (Blue-Green)
kubectl argo rollouts promote frontend -n hipster-frontend

# Abort rollout (revert to BLUE during promotion)
kubectl argo rollouts abort frontend -n hipster-frontend

# Undo rollout (rollback)
kubectl argo rollouts undo frontend -n hipster-frontend

# View history
kubectl argo rollouts history rollout frontend -n hipster-frontend
```

28.5 Docker Commands

```
# Build
docker build -t yampss/hipstershop-adservice:local src/

# Test locally
docker run -p 9555:9555 -e PORT=9555 yampss/hipstershop-adservice:local

# Check image layers
docker history yampss/hipstershop-adservice:adservice-v0.1.2

# Scan with Trivy
trivy image yampss/hipstershop-adservice:adservice-v0.1.2 \
  --severity CRITICAL --exit-code 1 --ignore-unfixed

# Cross-push to GHCR
docker tag yampss/hipstershop-adservice:adservice-v0.1.2 \
  ghcr.io/hipstershopp/hipstershop-adservice:adservice-v0.1.2
docker push ghcr.io/hipstershopp/hipstershop-adservice:adservice-v0.1.2
```

28.6 Health Check Endpoints

```
# Test all service health endpoints (run after port-forwarding)
curl http://localhost:9555/_healthz      # adservice
curl http://localhost:8081/_healthz     # authservice
curl http://localhost:7070/_healthz     # cartservice
curl http://localhost:5050/_healthz     # checkoutservice
curl http://localhost:7000/_healthz     # currencyservice
curl http://localhost:8080/_healthz     # emailservice / assistantservice /
recommendationservice
curl http://localhost:8080/_healthz     # frontend (port-forward svc/frontend 8080:80)
curl http://localhost:50051/_healthz    # paymentservice / shippingservice
curl http://localhost:3550/_healthz    # productcatalogservice
```

28.7 Service-Specific Test Calls

```
# authservice – signup and login
curl -X POST http://localhost:8081/signup \
  -H "Content-Type: application/json" \
  -d '{"email":"test@test.com","password":"test1234"}'

curl -X POST http://localhost:8081/login \
  -H "Content-Type: application/json" \
  -d '{"email":"test@test.com","password":"test1234"}'

# adservice – get ads
curl http://localhost:9555/ads

# productcatalogservice – list products
curl http://localhost:3550/products

# paymentservice – test charge
curl -X POST http://localhost:50051/charge \
  -H "Content-Type: application/json" \
  -d '{
    "amount":{"currencyCode":"USD","units":10,"nanos":0},
    "creditCard":{"
      "creditCardNumber":"4432-8015-6152-0454",
      "creditCardCvv":672,
      "creditCardExpirationYear":2029,
      "creditCardExpirationMonth":1
    }
  }'

# currencyservice – list currencies
curl http://localhost:7000/currencies

# assistantservice – chat
curl -X POST http://localhost:8080/api/assistant/chat \
  -H "Content-Type: application/json" \
  -d '{"productId":"0LJCESPC7Z0","message":"Tell me about this product"}'
```

Appendix: Sample values.yaml Structure

```
# hipstershop-micro-apps/{service}/values.yaml
global:
  namespaces:
    backend: hipster-backend
    frontend: hipster-frontend
    database: hipster-database

  images:
    adservice: "yampss/hipstershop-adservice:adservice-v0.1.2"
    authservice: "yampss/hipstershop-authservice:authservice-v0.2.1"
    # ... all 12 services

  replicas:
    adservice: 1
    authservice: 1
    # ...

  ports:
    adservice: 9555
    authservice: 8081
    cartservice: 7070
    checkoutservice: 5050
    currencyservice: 7000
    emailservice: 8080
    frontend: 8080
    paymentservice: 50051
    productcatalogservice: 3550
    recommendationservice: 8080
    shippingservice: 50051
    assistantervice: 8080

  rollingUpdate:
    maxUnavailable: 1
    maxSurge: 1

  resources:
    adservice:
      requests:
        cpu: 200m
        memory: 180Mi
      limits:
        cpu: 300m
        memory: 512Mi

  hpa:
    frontend:
```

```
    minReplicas: 1
    maxReplicas: 5
    cpuUtilization: 70
    memoryUtilization: 80

blueGreen:
  autoPromotionEnabled: false
  scaleDownDelaySeconds: 60

gateway:
  name: hipstershop-gateway
  port: 80

networkPolicy:
  vpcCidr: "172.31.0.0/16"
  kgatewayNamespace: "kgateway-system"
```

End of HipsterShop Platform — Complete DevOps Engineering Documentation

Document generated: April 2026 / Organization: HipsterShopp / 14 Repositories / 12 Microservices